

BernatLab

LoRa, sensors remots i xarxa de camp

Raspberry Pi · Docker · IA · LoRa · Hort Osona

Mòdul 3

Bernat — 8 de juliol del 2026

Document tècnic de consulta personal

Sobre aquest manual

Aquest és el segon mòdul del manual tècnic del BernatLab. Cobreix tota la cadena de sensors i visualització: el protocol MQTT, el broker Mosquitto, l'esquema de publicació dels sensors, la base de dades InfluxDB, l'agent Telegraf, la programació visual amb Node-RED, la visualització amb Grafana, l'API REST amb FastAPI, la integració amb la web pública Hort Osona i l'operativa del dia a dia.

Com es llegeix

En ordre, si parteixes de zero. Per capítols, si ja tens una base i vols aprofundir. Cada capítol segueix la mateixa estructura: teoria, aplicació al BernatLab, esquemes, comandes útils, errors habituals, exercicis pràctics i resum final.

Índex de capítols

- 11. Del Mòdul 1 al M2: què construïm
- 12. MQTT des de zero
- 13. Mosquitto al BernatLab
- 14. Publicar dades: els sensors
- 15. InfluxDB: base de dades de sèries temporals
- 16. Telegraf: el pont
- 17. Node-RED: programació visual
- 18. Fluxos pràctics
- 19. Grafana: visualitzar les dades
- 20. API pública: servir les dades al món
- 21. Integració amb Hort Osona
- 22. Operativa: còpies, alertes, escalat

Capítol 23 — Què és LoRa i per què per a Hort Osona

"245 metres, arbres al mig, cases al voltant, Wi-Fi de veïns. Si vols que un sensor al camp parli amb la Raspberry, la resposta pràctica és LoRa."

23.1 El problema: distància i obstacles

Al BernatLab, tenim sensors al camp (l'hort és a uns 245 metres de casa) que han de comunicar-se amb la Raspberry. Les opcions "naturals" fallen totes:

- **Wi-Fi domèstica:** té un abast útil de 30-50 metres a l'aire lliure, menys a través de parets i arbres. A 245 metres, amb vegetació al mig, és pràcticament impossible sense repetidors o antenes directives.
- **Bluetooth/BLE:** 10-30 metres, insuficient.
- **Zigbee:** 100 metres en exterior, depèn molt del terreny. Millor que Wi-Fi, però encara just.
- **4G/cel·lular:** funciona, però requereix una SIM, una subscripció, i un mòdem. És car, depèn de cobertura, i afegeix un component de xarxa mòbil que complica el projecte.

Cap d'aquests encaixa bé. I aquí és on entra **LoRa**: una tecnologia de ràdio dissenyada exactament per cobrir distàncies d'uns quants quilòmetres amb un consum d'energia molt baix. LoRa no és una novetat — la tecnologia es remunta al 2015, quan Semtech va obrir les patents per a ús civil a la banda sub-GHz — però en els últims anys s'ha convertit en l'estàndard de facto per a IoT de llarg abast i baix consum.

23.2 Què és LoRa, exactament

LoRa (de **Long Range**) és una tècnica de **modulació de ràdio** propietat de Semtech, que opera a les bandes ISM (Industrial, Scientific, Medical) — bandes de freqüència lliures, sense necessitat de llicència, però subjectes a normatives locals de potència i ocupació.

Concretament, al BernatLab i a la majoria d'Europa, LoRa opera a la **banda dels 868 MHz** (de 863 a 870 MHz, amb sub-bandes específiques per a diferents usos). Aquesta banda té tres propietats excel·lents per al nostre cas:

1. **Penetra obstacles** molt millor que les freqüències altes (2,4 GHz del Wi-Fi, per exemple). Una ona a 868 MHz travessa arbres, parets de maó, i fins i tot edificis lleugers amb molta menys pèrdua.
2. **Consum molt baix:** la modulació és eficient, cosa que permet transmissions amb pocs mil·liwatts de potència.
3. **Lliure d'ús:** no cal pagar llicència, només respectar la normativa de duty cycle (percentatge de temps que podem transmetre).

LoRa, per si sol, és només una **modulació física** (PHY). Defineix com es modula el senyal, però no com s'organitzen les dades, com es xifra, com es gestiona l'accés al canal, com es connecten múltiples nodes. Per a tot això, necessitem una **capa superior** — i aquí és on apareix **LoRaWAN**.

- **Integracions fàcils:** podem rebre les dades a la Raspberry per MQTT, HTTP webhook, o altres mecanismes.

L'alternativa open source és **ChirpStack**, que podem autoallotjar a la Raspberry o a un servidor propi. Té avantatges (control total, sense limits de la comunitat) i inconvenients (més feina de manteniment, hem de gestionar la seguretat nosaltres mateixos).

23.5 LoRa P2P: l'alternativa sense infraestructura

Hi ha una tercera via: usar la **modulació LoRa directament, sense LoRaWAN**. Això es coneix com a **LoRa punt a punt (P2P)** o **LoRa raw**. En aquest model:

- Dos mòduls SX1262 parlen directament entre ells, sense gateway, sense network server.
- Nosaltres definim el protocol: la longitud del payload, la freqüència, el spreading factor, si hi ha ACK o no.
- És més simple, però perdem tota la infraestructura de xarxa de LoRaWAN: no podem afegir un segon node fàcilment, no tenim deduplicació, no tenim xifratge estandarditzat.

Quan té sentit LoRa P2P?

- Quan tenim un sol node i un sol gateway casolà, i no volem la complexitat de TTN.
- Quan estem experimental i volem veure "com rau" LoRa abans d'introduir cap capa superior.
- Quan tenim limitacions de volum o privacitat que no ens permeten usar TTN.

Quan NO té sentit LoRa P2P?

- Quan volem afegir més d'un node.
- Quan volem un sistema escalable.
- Quan volem compatibilitat amb estàndards.

Al BernatLab, tenim clar que voldrem diversos nodes a l'hort (temperatura, humitat del sòl, llum, pluja...) i potser un node extra al camp del veí. Per tant, **LoRaWAN és la via natural**.

Dit això, durant el capítol 31 veurem un exemple de LoRa P2P, per si mai el necessites per a una situació específica o simplement per aprendre els fonaments de la capa física.

23.6 Quin model triem per a Hort Osona

L'arbre de decisió:

```

Estàs disposat a usar serveis de tercers (TTN, Helium)?
■■■ Sí → LoRaWAN amb TTN
■      ■■■ Tens cobertura d'un gateway TTN comunitari a menys de 5 km?
■      ■    ■■■ Sí → Usa el gateway comunitari, no cal comprar res
■      ■    ■■■ No → Compra o munta un gateway propi
■      ■■■ Tens cobertura d'un gateway comercial (Loriot, Actility)?
■      ■    ■■■ Sí → Considera aquesta opció (pot ser de pagament)
■      ■    ■■■ No → Mateixa resposta: gateway propi
■■■ No → LoRa P2P amb SX1262
        ■■■ Tens 1 o 2 nodes i poca escalabilitat
  
```

Per a Hort Osona, la recomanació és:

1. **Comprovar si hi ha cobertura TTN** a la zona. Pots fer-ho a <https://ttnmapper.org/> o obrint la consola de TTN i mirant els gateways propers. A Osona (Vic, Manresa, àmbit rural català), la cobertura TTN comunitària és **decent però no omnipresent**. Si n'hi ha un a 3-5 km amb visió directa, podem començar a usar-lo.
2. **Si no n'hi ha**, muntem un gateway propi a la Raspberry. El programari estàndard és **Concentratord** (abans anomenat lora-packet-forwarder), de xdegaye, i és el que recomanarem al capítol 27.
3. **Els nodes** seran ESP32 + SX1262, amb la llibreria **LMIC** (IBM's LoRa MAC in C) o **MCCI LoRaWAN LMIC** per a LoRaWAN. Per a P2P, la llibreria **RadioLib** de jgromes.

23.7 Components que necessitem

A grans trets, la llista de la compra per a Hort Osona:

Per al gateway (a la Raspberry)

- Un **mòdul concentrador LoRa**: pot ser un **RAK2287** (USB, amb un SPI intern), un **Waveshare SX1302 LoRaWAN Gateway HAT** (per a Raspberry, molt popular), o un **Dragino LPS8** (gateway comercial, més car però robust).
- Una **antena 868 MHz** amb connector compatible.
- Un **cable pigtail** SMA si la distància entre el mòdul i l'antena és gran.

Per als nodes (al camp)

- Un **ESP32** (molta varietat: WROOM, WROVER, DevKit, etc.).
- Un **mòdul SX1262**: pot ser en forma de **breakout board** (per a prototipatge en protoboard) o integrat en plaques com **Heltec LoRa 32** o **TTGO LoRa**.
- **Sensors** específics del que vulguem mesurar.
- Una **antena 868 MHz** adequada per a la banda.
- **Bateria o placa solar** per a ús autònom.

Per al programari

- **The Things Stack** (consola web, gratuït).
- **Concentratord** (a la Raspberry) si no hi ha gateway TTN proper.
- **Node-RED** o un script Python per rebre els missatges via MQTT.

23.8 Com connecta amb la resta del BernatLab

El pipeline complet, un cop LoRa estigui en marxa:

Sensor al camp (ESP32 + SX1262)

■ (LoRaWAN o LoRa P2P)



Gateway (Concentratord a la Raspberry o TTN)

■ (MQTT sobre TLS)



Network Server (TTN o autoallotjat)

■ (MQTT, o webhook, o directament a Telegraf)



BernatLab (Raspberry)

■■■ Mosquitto (broker MQTT)

■■■ Telegraf (recol·lector)

■■■ InfluxDB (base de dades)

■■■ Node-RED (processament)

■■■ Grafana (visualització)

■■■ API FastAPI (per a la web Hort Osona)

Cada element ja està cobert als mòduls anteriors. El Mòdul 3 afegeix només la primera peça: **el node i el gateway LoRa**, i la integració amb el broker MQTT que ja tenim.

23.9 Esquema conceptual

Esquema Mermaid (text):

```
graph TB
    subgraph Hort["Hort Osona (245 m)"]
        N1["Node 1: T/H/llum<br/>(ESP32 + SX1262)"]
        N2["Node 2: humitat sòl"]
        N3["[futur: altres nodes]"]
    end

    subgraph Radio["Capa ràdio (868 MHz)"]
        SIG["Senyal LoRa<br/>SF7-SF12"]
    end

    subgraph Casa["Casa (Raspberry Pi 4)"]
        GW["Gateway<br/>(Concentrator)"]
        MOSQ["Mosquitto"]
        TEL["Telegraf"]
        INF["InfluxDB"]
    end

    subgraph NS["Network Server"]
        TTN["The Things Stack<br/>(o ChirpStack local)"]
    end

    N1 --> SIG
    N2 --> SIG
    N3 -. -> SIG
    SIG --> GW
    GW --> TTN
    TTN -->|MQTT| MOSQ
    MOSQ --> TEL
    TEL --> INF
```

23.10 Glossari de termes LoRa

Abans de continuar, fixem els termes que farem servir:

- **LoRa**: tècnica de modulació de ràdio (PHY).
- **LoRaWAN**: protocol de xarxa sobre LoRa (MAC + aplicació).
- **End Device / Node**: el sensor, amb ràdio LoRa.
- **Gateway**: aparell que rep transmissions LoRa i les passa al network server.
- **Network Server (NS)**: gestiona la xarxa LoRaWAN.
- **Application Server (AS)**: rep les dades ja desxifrades.

- **DevEUI / AppEUI / AppKey**: identificadors del node (veure capítols següents).
- **OTAA / ABP**: els dos mètodes d'unió d'un node a la xarxa.
- **SF (Spreading Factor)**: paràmetre de la modulació, de SF7 (ràpid, curt abast) a SF12 (lent, llarg abast).
- **BW (Bandwidth)**: amplada de banda del canal (125, 250, 500 kHz típicament).
- **RSSI / SNR**: indicadors de qualitat del senyal.
- **TTN / TTS**: The Things Network / The Things Stack.
- **ADR (Adaptive Data Rate)**: mecanisme pel qual el network server ajusta els paràmetres de transmissió segons la qualitat del senyal.

23.11 Què aprendrem en aquest mòdul

Aquest mòdul cobreix, en 10 capítols:

- **Cap 24** — Física de ràdio: freqüència, amplada de banda, spreading factor, RSSI, SNR, duty cycle, normativa.
- **Cap 25** — LoRaWAN vs LoRa P2P: avantatges, inconvenients, decisió.
- **Cap 26** — La capa LoRaWAN: TTN, device profiles, formats de payload, decoders.
- **Cap 27** — Muntar un gateway LoRaWAN a la Raspberry amb Concentratord.
- **Cap 28** — Hardware del node: ESP32 + SX1262, esquema elèctric, antena, alimentació.
- **Cap 29** — Programar el node: ESP32 amb Arduino, codi complet d'un sensor LoRaWAN.
- **Cap 30** — Recepció al BernatLab: TTN → Mosquitto → Telegraf → InfluxDB.
- **Cap 31** — LoRa P2P: cas alternatiu per a un sol node, exemple complet amb RadioLib.
- **Cap 32** — Proves de camp, cobertura, calibratge, resolució de problemes.
- **Cap 33** — Mòdul 3 a la pràctica: el teu primer node LoRa en producció.

23.12 Resum

En aquest capítol hem vist què és LoRa (modulació de ràdio de llarg abast a 868 MHz) i per què és la tecnologia adequada per a Hort Osona: distància, penetració d'obstacles, baix consum, sense llicència. Hem après l'arquitectura de LoRaWAN (node → gateway → network server → application server) i hem decidit que la millor opció per al nostre cas és **LoRaWAN amb The Things Stack**, amb un gateway propi si no n'hi ha cap comunitari a prop. Hem vist quin hardware cal i com es connecta amb la resta del BernatLab. En el proper capítol ens endinsarem en la física de ràdio: freqüència, amplada de banda, spreading factor, i la resta de paràmetres que defineixen una transmissió LoRa.

23.13 Exercicis pràctics

1. Comprova si hi ha cobertura TTN a la teva zona: <https://ttnmapper.org/>
2. Crea un compte a The Things Network: <https://console.thingsnetwork.org/>

3. Comprova si tens visió directa entre casa i l'hort. Si no, identifica on podries posar el gateway.
4. Fes una llista del hardware que necessitaries comprar: gateway, nodes, antenes, sensors, bateries.
5. Comprova el preu de: - Un gateway Waveshare SX1302 per a Raspberry Pi (~50-80 €). - Un node Heltec LoRa 32 V3 (~25 €). - Un node ESP32-DevKit + SX1262 breakout (~15-25 €).
6. Documenta al README del projecte quina estratègia has decidit (TTN + gateway propi, P2P, etc.).

Paraules clau: **LoRa, LoRaWAN, 868 MHz, ISM, gateway, network server, application server, The Things Network, TTN, ChirpStack, Concentrator, packet forwarder, SX1262, ESP32, RSSI, SNR, spreading factor, bandwidth, node, end device, OTAA, ABP, ADR, duty cycle, EU868, MQTT, Telegraf, InfluxDB, BernatLab, Hort Osona, RPi 4, cobertura, visió directa, antena, pigtail, breakout, dev kit, Heltec, TTGO, LMIC, RadioLib, Arduino, MicroPython, ESP-IDF.**

Capítol 24 — Física de ràdio: els paràmetres que importen

"LoRa és com una guitarra: amb els mateixos materials pots fer un concert o un mal so. La diferència està en com toques les cordes."

24.1 Què passa quan un mòdul LoRa transmet

Abans d'entendre els paràmetres, cal entendre què està passant realment a nivell físic. Quan un SX1262 transmet, fa el següent:

1. **Codifica les dades** en bits.
2. **Aplica un codi de correcció d'errors** (FEC, Forward Error Correction). LoRa usa un codi amb taxa variable (de 4/5 a 4/8) — com més protecció, menys capacitat però més robustesa.
3. **Intercala els bits** per protegir-se de ràfegues d'errors.
4. **Modula amb Chirp Spread Spectrum (CSS)**: cada bit es converteix en un "chirp", un senyal que canvia de freqüència al llarg del temps de manera lineal. La freqüència instantània augmenta (up-chirp) o disminueix (down-chirp) contínuament.
5. **Emet el senyal** a la freqüència portadora (868 MHz típicament) amb una amplada de banda (BW) concreta i un spreading factor (SF) determinat.

A la recepció, el procés és invers: el receptor correlaciona el senyal rebut amb chirps coneguts i n'extreu els bits originals.

La bellesa d'aquest esquema és que **com més ample de banda i més spreading factor, més robust** és el senyal davant de soroll, interferències i obstacles — però més **lent** és. La capacitat de transmissió i la sensibilitat del receptor es poden ajustar amb només tres paràmetres: BW, SF i CR.

24.2 Amplada de banda (Bandwidth, BW)

L'**amplada de banda** és el rang de freqüències que el senyal ocupa. LoRa estandarditza tres valors principals:

- **125 kHz**: el més comú, bon equilibri capacitat/abast. Usa la meitat del canal.
- **250 kHz**: més capacitat, però ocupa tot un canal i consumeix més energia.
- **500 kHz**: el màxim, molt capacitat però poca eficiència energètica.

A la banda EU868, tenim canals definits a 868.1, 868.3, 868.5, 867.1, 867.3, 867.5, 867.7, 867.9 MHz (8 canals estàndard a 125 kHz). Per a 250 kHz, hi ha canals a 868.5, 867.1, 867.7. Per a 500 kHz, a 868.5 MHz.

Quan fer servir cada BW?

- **125 kHz**: per defecte. Sempre que no necessitem més capacitat.
- **250 kHz**: quan volem enviar més dades (firmware updates, payloads grans).
- **500 kHz**: només per experiments; rarament útil.

24.3 Spreading Factor (SF)

El **spreading factor** és el paràmetre més important. Determina quant s'expandeix el senyal en el temps. LoRa defineix SF7 a SF12, amb SF7 el més ràpid i SF12 el més robust:

SF	Bits/simbòlic	Temps per byte	Sensibilitat (aprox.)	Abast relatiu
SF7	7	41 ms	-123 dBm	1× (base)
SF8	8	72 ms	-126 dBm	1.4×
SF9	9	144 ms	-129 dBm	2×
SF10	10	288 ms	-132 dBm	2.8×
SF11	11	577 ms	-134.5 dBm	4×
SF12	12	992 ms	-137 dBm	5.7×

Com interpretar-ho?

- A SF7, podem enviar un missatge en 41 ms i el receptor detecta senyals fins a -123 dBm (un nivell molt baix). Però cada byte "costa" poca energia de senyal.
- A SF12, el mateix missatge triga 24 vegades més (992 ms), però el receptor detecta senyals 14 dB més febles, equivalent a 5-6 vegades més distància.

A la pràctica: per a sensors ambientals amb pocs bytes cada 5-15 minuts, **SF7 o SF8** funcionen perfectament a 245 metres amb bona visió. Si tenim obstacles o volem més marge, **SF9-SF11**. **SF12** és per a casos extrems (quilòmetres) o quan tenim pèrdues molt severes.

Quin és el cost? A SF12, **cada transmissió gasta 24 vegades més temps d'aire** que a SF7. A la banda EU868, hi ha un límit de **duty cycle** (1% per defecte): només podem transmetre l'1% del temps. Si un missatge a SF12 dura 1 segon, hem d'esperar 99 segons entre missatges. A SF7, podem enviar un missatge cada ~4 segons.

24.4 Coding Rate (CR)

El **coding rate** (4/5, 4/6, 4/7, 4/8) defineix la redundància afegida pel codi de correcció d'errors. Com més baix, menys protecció però menys overhead. Per defecte, 4/5 és el valor estàndard. Si tenim molts errors, podem pujar a 4/7 o 4/8.

24.5 Cicle de treball (Duty Cycle)

A la banda EU868, la normativa ETSI EN 300 220 limita el **duty cycle** (percentatge de temps que podem transmetre) a:

- Sub-banda 868.0-868.6 MHz: **1%** (és a dir, com a molt 36 segons per hora).
- Sub-banda 868.7-869.2 MHz: **0.1%** (12 segons per hora).
- Sub-banda 869.4-869.65 MHz: **10%** (potència limitada a 500 mW).

A la pràctica, la majoria de xarxes LoRaWAN usen la sub-banda 868.0-868.6 amb 1% de duty cycle. Això vol dir que si un missatge dura 1 segon, hem d'esperar 99 segons fins al proper. Per a sensors ambientals, és perfecte: publiquem cada 5-15 minuts, molt per sota del límit.

Però compte: el càlcul és per sub-banda, no per node. Si tenim 5 nodes transmetent a la mateixa sub-banda, tots compten contra el 1% agregat. En sistemes petits, no és un problema. En sistemes grans, cal planificar.

24.6 Potència de transmissió (TX Power)

LoRa permet ajustar la potència entre **2 dBm (1.6 mW)** i **20 dBm (100 mW)** típicament, segons el mòdul. Com més potència, més abast però més consum de bateria.

Per a un sensor a 245 metres amb bona visió, **8-10 dBm (6-10 mW)** és més que suficient. Per a distàncies quilomètriques o amb obstacles, pujar a **14-17 dBm (25-50 mW)**.

A la Raspberry (gateway), solem usar **14-17 dBm**, que és el que solen suportar els mòduls concentradors SX1302.

Important: la potència màxima legal a EU868 és **25 mW ERP (14 dBm)** per a la majoria de sub-bandes, tot i que el SX1262 pot arribar a 20 dBm. Si anem més enllà del límit legal, podem tenir problemes amb les autoritats de telecomunicacions (a Espanya, la Secretaría de Estado de Telecomunicaciones).

24.7 RSSI i SNR

Quan el gateway rep un missatge, ens dóna dues mètriques de qualitat:

- **RSSI (Received Signal Strength Indicator)**: la potència del senyal rebut, en dBm. Com més proper a 0, millor. Valors típics:
 - **-30 a -60 dBm**: senyal excel·lent (a pocs metres).
 - **-60 a -90 dBm**: senyal bo (a desenes de metres).
 - **-90 a -110 dBm**: senyal acceptable (a cent metres).
 - **-110 a -120 dBm**: senyal just (a quilòmetres, depèn de l'entorn).
 - **< -120 dBm**: massa feble per a SF7, pot funcionar a SF12.
- **SNR (Signal-to-Noise Ratio)**: la relació senyal-soroll, en dB. Com més alt, millor. Valors típics:
 - **> 10 dB**: excel·lent.
 - **5-10 dB**: bo.
 - **0-5 dB**: acceptable.
 - **< 0 dB**: el senyal està per sota del soroll; la modulació LoRa encara pot desxifrar-lo gràcies a la seva robustesa, però estem al límit.

Com usar RSSI i SNR a la pràctica?

A TTN (i a la majoria de network servers), podem veure RSSI i SNR de cada missatge. Si veiem que RSSI baixa gradualment, pot ser que la bateria del node s'està acabant. Si SNR baixa sobtadament, pot ser que un obstacle nou (una branca, una persona) estigui bloquejant el senyal.

24.8 L'equació de la capacitat: bit rate, time on air, payload màxim

LoRa té una fórmula per calcular el **bit rate** i el **time on air** d'un missatge. Sense entrar en les matemàtiques completes, les conclusions pràctiques són:

- A **SF7, BW 125 kHz**: el bit rate és ~5.5 kbps, i el temps d'un missatge de 20 bytes és ~60 ms.
- A **SF12, BW 125 kHz**: el bit rate és ~0.3 kbps, i el mateix missatge dura ~1.5 segons.

El **payload màxim** (mida útil de dades per transmissió) varia amb SF:

- SF7: 222 bytes.
- SF8: 222 bytes.
- SF9: 115 bytes.
- SF10: 51 bytes.
- SF11: 51 bytes.
- SF12: 51 bytes.

Per a sensors ambientals (temperatura, humitat, etc.), el payload és típicament 5-20 bytes, lluny dels límits.

24.9 ADR: Adaptive Data Rate

ADR és un mecanisme pel qual el **network server** ajusta els paràmetres de transmissió (SF, BW, potència) de cada node en funció de la qualitat del senyal que rep. La idea és:

- Si un node s'envia amb SF12 però el gateway el rep a RSSI -50 dBm, el node està gastant molta energia innecessàriament. El NS li pot dir "baixa a SF8 i a 8 dBm".
- Si un node s'envia amb SF7 però el gateway el rep a RSSI -115 dBm, el NS li pot dir "puja a SF10".

L'ADR és opcional, però molt recomanable. Millora la capacitat de la xarxa i allarga la vida de les bateries. TTN l'implementa per defecte.

24.10 Les bandes ISM al món

Cada regió té les seves bandes ISM. Les més importants:

- **EU868** (Europa, Àfrica, bona part d'Àsia): 863-870 MHz, amb sub-bandes específiques. La que ens afecta.
- **US915** (EUA): 902-928 MHz, amb 64 canals a 125 kHz (més canals, però menys temps d'aire per missatge).
- **AS923** (Àsia-Pacífic): 915-928 MHz, una mena de pont entre EU868 i US915.
- **AU915** (Austràlia): similar a US915.
- **CN470** (Xina): 470-510 MHz, diferent de les altres.
- **IN865** (Índia): 865-867 MHz.

- **KR920** (Corea): 920-925 MHz.

Al BernatLab, estem a Europa, per tant treballem amb **EU868**.

24.11 El concepte de "canal" i "data rate"

A LoRaWAN, una combinació de SF + BW es coneix com a **Data Rate (DR)**. Per exemple:

- **DR0**: SF12, BW 125 kHz (més lent, més abast).
- **DR5**: SF7, BW 125 kHz (més ràpid, menys abast).

A EU868, els data rates van de DR0 a DR5 (6 valors). La idea és la mateixa que amb SF aïllat: valors baixos = més abast, valors alts = més velocitat.

Quan parlem de "canal", ens referim a una freqüència concreta dins de la banda. Un missatge concret es transmet en un canal concret amb un data rate concret.

24.12 RSSI vs SNR vs Link Budget

El **link budget** és la diferència entre la potència transmesa i la sensibilitat del receptor. Per exemple:

- TX Power: 14 dBm.
- Sensibilitat del receptor (a SF7): -123 dBm.
- Link budget: 14 - (-123) = 137 dB.

Aquest número ens diu quanta "pèrdua" podem tolerar entre l'antena emissora i la receptora. A 868 MHz, la pèrdua en espai lliure segueix la fórmula de Friis:

$$P_r(\text{dBm}) = P_t(\text{dBm}) + G_t(\text{dBi}) + G_r(\text{dBi}) - 20 \cdot \log_{10}(d) - 20 \cdot \log_{10}(f) - 32.45$$

on d és la distància en km i f la freqüència en MHz. A 868 MHz, 1 km, 0 dBi d'antenes:

$$P_r = 0 - 20 \cdot \log_{10}(1) - 20 \cdot \log_{10}(868) - 32.45 \approx -91.2 \text{ dBm}$$

A 245 metres, afegint-hi 0 dBi d'antenes (cas base):

$$P_r = 0 - 20 \cdot \log_{10}(0.245) - 20 \cdot \log_{10}(868) - 32.45 \approx -66.8 \text{ dBm}$$

Això és molt millor que -123 dBm (sensibilitat a SF7), per tant SF7 funciona bé en espai lliure a 245 metres. Si afegim obstacles (un arbre de 5 metres al mig), la pèrdua pot pujar a 10-20 dB, i estarem a -85 dBm, encara perfectament dins del rang.

24.13 Què vol dir "Banda ISM"

ISM (Industrial, Scientific, Medical) són bandes de freqüència reservades per a ús industrial, científic i mèdic sense necessitat de llicència. N'hi ha diverses al món:

- 6.78 MHz, 13.56 MHz, 27.12 MHz (ús general).
- 433.92 MHz (Europa, Àsia).
- 868 MHz (Europa, Àsia).
- 915 MHz (Amèrica).
- 2.45 GHz (Wi-Fi, Bluetooth, microones).

- 5.8 GHz (Wi-Fi, alguns radars).

Aquestes bandes estan compartides amb molts altres serveis, per la qual cosa la normativa imposa **límits de potència i duty cycle** per evitar interferències.

24.14 Què passa si comparteixo la banda amb altres

A 868 MHz, podem trobar:

- Sistemes d'alarma sense fils.
- Lectors RFID.
- Comandaments a distància de portes de garatge.
- Sensors industrials.
- Altres xarxes LoRa.

Com que la modulació LoRa és molt robusta (gràcies al spreading factor i al CSS), aguanta bé la coexistència amb altres sistemes. Tanmateix, podem rebre **interferències**, que es manifesten com a SNR baix o paquets perduts.

Solucions:

- Canviar de canal (si la nostra xarxa ho permet).
- Usar SF més alt (més robust davant d'interferències, però més lent).
- Usar ADR perquè el NS trobi la millor combinació automàticament.

24.15 Glossari visual

Esquema Mermaid (text):

```
graph TB
    subgraph Parametres ["Paràmetres de transmissió LoRa"]
        SF["Spreading Factor<br/>(SF7 - SF12)"]
        BW["Bandwidth<br/>(125/250/500 kHz)"]
        CR["Coding Rate<br/>(4/5 - 4/8)"]
        TX["TX Power<br/>(2-20 dBm)"]
    end

    subgraph Sortida ["Resultats"]
        AIR["Time on Air<br/>(durada transmissió)"]
        SENS["Sensibilitat<br/>(potència mínima detectable)"]
        BR["Bit Rate<br/>(kbps)"]
    end

    subgraph Qualitat ["Qualitat del senyal rebut"]
        RSSI2["RSSI (potència)"]
        SNR2["SNR (relació senyal/soroll)"]
        PL["Packet Loss<br/>(% de paquets perduts)"]
    end

    SF --> AIR
    SF --> SENS
    BW --> AIR
    BW --> BR
    CR --> AIR
    TX --> AIR

    AIR --> PL
    SENS --> PL
```

BR --> PL

AIR --> RSSI2
SENS --> SNR2

24.16 Com aplicar-ho a Hort Osona

Per a un node a 245 metres de la Raspberry, amb possibles arbres al mig, una bona configuració inicial és:

- **BW:** 125 kHz.
- **SF:** SF9 (bon equilibri abast/consum/velocitat).
- **CR:** 4/5 (per defecte).
- **TX Power:** 10 dBm (10 mW) — abundant per a 245 m, deixa marge.
- **ADR:** activat (TTN el gestionarà sol).

Aquesta configuració permet transmissions cada 5-10 minuts, payload de fins a 115 bytes, i una vida útil de bateria de mesos amb piles AA o una LiPo petita.

24.17 Resum

En aquest capítol hem après la física darrere de LoRa: com la modulació Chirp Spread Spectrum permet transmissions de llarg abast amb poc consum, què significa cada paràmetre (BW, SF, CR, TX Power), com es mesura la qualitat del senyal (RSSI, SNR), quines limitacions normatives tenim (duty cycle, potència màxima), i quines combinacions funcionen bé per a l'escenari de Hort Osona. En el proper capítol veurem les dues grans topologies: LoRaWAN i LoRa P2P, i decidirem quina és la millor per al nostre cas.

24.18 Exercicis pràctics

1. Calcula el temps on air d'un missatge de 20 bytes a SF7, BW 125 kHz, CR 4/5. Usa una calculadora online com <https://www.loratools.nl/#/airtime>.
2. Calcula el temps on air del mateix missatge a SF12. Quantes vegades més llarg és?
3. Comprova quins canals de 868 MHz estan lliures a la teva zona amb un scanner com <https://github.com/cyberman54/LoRa-Scanner>.
4. Si el teu node envia cada 5 minuts (300 segons) un missatge de 100 ms a SF7, quin percentatge de duty cycle uses? (Pista: $100 \text{ ms} / 300 \text{ s} = 0,03\%$.)
5. Quina seria la durada màxima de missatge a SF12, BW 125 kHz, sense superar el 1% de duty cycle a la sub-banda principal?
6. Comprova la pèrdua en espai lliure a 245 metres, 868 MHz, amb la fórmula de Friis.
7. Fes una taula amb les combinacions BW × SF que faries servir per a: - Sensor a 50 metres (interior de casa). - Sensor a 245 metres (hort). - Sensor a 1 km (per si mai).

Paraules clau: **LoRa, modulació, chirp, CSS, spreading factor, SF7, SF8, SF9, SF10, SF11, SF12, bandwidth, BW, 125 kHz, 250 kHz, 500 kHz, coding rate, duty cycle, ETSI EN 300 220, EU868, ISM, RSSI, SNR, packet loss, link budget, Friis, pèrdua en espai lliure, ADR, data rate,**

canal, payload, time on air, antena, dBi, dBm, mW, potència, normativa, telecomunicacions, 868 MHz, 915 MHz, US915, EU868, AS923, TTN, TTN, xarxa, network server, node, gateway, end device, OTAA, ABP, EU868, EU433, US915, CN470, KR920, IN865, AS923, AU915, ISM, sub-banda, 868.0, 868.3, 868.5, 867.1, 867.3, 867.5, 867.7, 867.9, 869.4, 869.65.

Capitol 25 — LoRaWAN vs LoRa P2P: l'arbre de decisió

"Hi ha dues maneres de fer anar un missatge per ràdio: amb un director d'orquestra (LoRaWAN) o tots sols (P2P). Les dues funcionen; només cal saber quan toca cada una."

25.1 Per què hem de triar

Quan vam al capítol 23 ja vam avançar la conclusió: per a Hort Osona, la millor opció és **LoRaWAN**. Però abans d'invertir temps i diners en una direcció, val la pena entendre les dues opcions a fons, per què cadascuna és millor en el seu context, i quines implicacions pràctiques té cadascuna.

Aquest capítol és una **decisió informada**. Si ja tens clar que vols LoRaWAN, pots saltar al capítol 26. Si tens dubtes, o vols entendre les alternatives, llegeix-lo sencer.

25.2 Recapitulació: què és LoRa i què és LoRaWAN

Ho vam veure al capítol 23, però ho repassem per tenir-ho fresc:

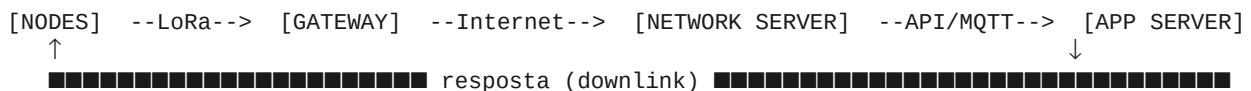
- **LoRa**: tecnologia de **modulació de ràdio** (PHY). Defineix com es modula el senyal a nivell físic. És propietat de Semtech.
- **LoRaWAN**: protocol de **xarxa** (MAC + aplicació) definit per la LoRa Alliance. Defineix com s'organitzen les dades, com s'autentiquen els nodes, com es xifra la comunicació, etc.

Això és similar a la distinció entre **Wi-Fi** (PHY) i **TCP/IP** (xarxa). Tots dos poden existir separatament, però combinats formen una xarxa completa.

25.3 LoRaWAN: l'opció "amb infraestructura"

Com funciona

Un node LoRaWAN no parla directament amb el gateway. Emet a l'aire, i qualsevol gateway que senti el senyal redirigeix el paquet al **network server**. El network server autentica el node, desxifra el missatge, i l'envia a l'**application server** (que pot ser TTN, ChirpStack, o un altre).



Quan volem enviar una ordre al node (per exemple, "obre la vàlvula de reg"), el flow és invers: app server → network server → gateway → node.

Avantatges

1. **Estàndard obert mantingut per una aliança industrial.** Compatibilitat entre fabricants, eines comunes, bona documentació.
2. **Network server gratuït o de baix cost.** TTN (The Things Stack) és gratuït per a ús no comercial. ChirpStack és open source i el podem autoallotjar.
3. **Escalabilitat.** Podem afegir tants nodes com vulquem, gestionats tots des d'una consola web.

4. **Seguretat integrada.** Xifratge AES-128, autenticació per DevEUI/AppKey, deduplicació, anti-replay.
5. **ADR automàtic.** El network server ajusta els paràmetres per optimitzar cada node.
6. **Downlink.** Podem enviar missatges al node (encara que amb limitacions de duty cycle).
7. **Mobilitat.** Si un node es mou d'un gateway a un altre, el network server redirigeix automàticament (roaming).
8. **Comunitat gran.** Milers de tutorials, fòrums, exemples de codi.
9. **Integracions fàcils.** TTN té integracions natives amb MQTT, webhooks, IFTTT, AWS, Azure, etc.

Inconvenients

1. **Compleixitat inicial.** Cal entendre DevEUI, AppEUI, AppKey, OTAA vs ABP, classes A/B/C, ADR, etc. La corba d'aprenentatge no és negligible.
2. **Dependència d'un network server.** Si TTN cau o canvia les seves polítiques, ens quedem penjats (menys problema amb ChirpStack autoallotjat).
3. **Limits de volum a TTN.** Per a ús gratuït, hi ha un límit de missatges per dia (uns 500-1000, segons la regió).
4. **Duty cycle europeu.** Només podem transmetre l'1% del temps a la majoria de sub-bandes, cosa que limita la freqüència d'actualitzacions.
5. **Cost del gateway.** Si no n'hi ha cap comunitari a prop, hem de comprar o muntar un gateway propi (50-300 €).

25.4 LoRa P2P: l'opció "sense infraestructura"

Com funciona

Dos mòduls SX1262 (o compatibles) es comuniquen directament entre ells, sense gateway, sense network server. Nosaltres definim tot: la freqüència, el SF, el payload, l'estructura del missatge, si hi ha ACK o no, etc.

```
[NODES] --LoRa--> [RX/TX mòdul] --cable/serial--> [Raspberry]
```

En aquest cas, la Raspberry (o un PC) té un mòdul SX1262 connectat per **SPI** o **USB** (amb un adaptador), i un script Python o C++ escolta els missatges rebuts.

Avantatges

1. **Simplicitat.** No cal configurar DevEUI, AppEUI, AppKey, ni entendre les classes de LoRaWAN. Tu tries el format del payload.
2. **Sense dependència de tercers.** Tot és nostre, no hi ha network server extern.
3. **Sense limits de volum.** Podem transmetre tan sovint com el duty cycle ens permeti.
4. **Cost inicial més baix.** Només cal el node i un mòdul SX1262 a la Raspberry. No cal gateway separat.
5. **Aprenentatge.** Codi obert, podem entendre exactament què passa.

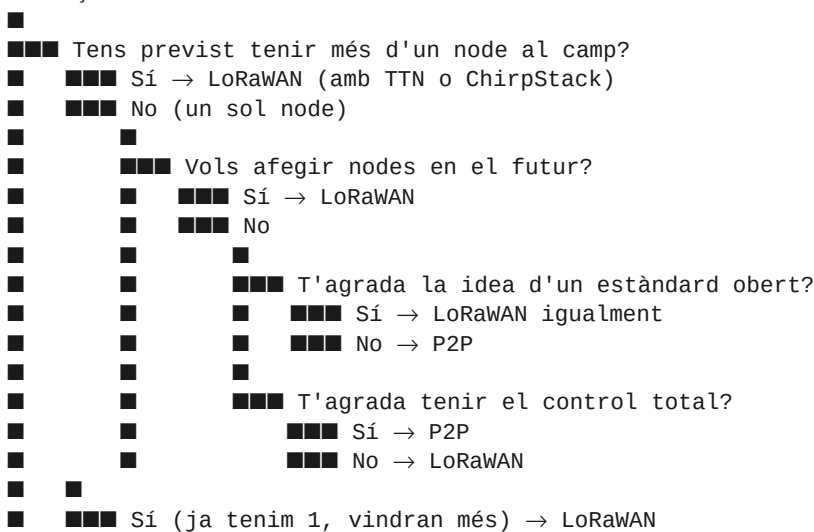
Inconvenients

1. **Sense estandardització.** Cada projecte fa el seu propi protocol, la qual cosa significa més feina de manteniment.
2. **Sense seguretat per defecte.** Si volem xifrar, l'hem d'implementar nosaltres.
3. **Sense escalabilitat.** Si volem afegir un segon node, hem de modificar el codi del receptor. Si volem un tercer, igual. Per a més de 2-3 nodes, el P2P esdevé caòtic.
4. **Sense roaming.** Si un node canvia de posició, ens podem perdre.
5. **Sense ADR.** Hem de configurar manualment els paràmetres de cada node.
6. **Sense downlink estandarditzat.** Podem implementar ACK manualment, però no és tan net com el downlink de LoRaWAN.
7. **Compatibilitat limitada.** Si volem afegir una cosa comercial (un sensor de tercers que parli LoRaWAN), no es comunicarà amb el nostre P2P.

25.5 Quin model per a Hort Osona

L'arbre de decisió complet:

Comença



Per a Hort Osona, la resposta és clara: **LoRaWAN**. Tenim previst diversos nodes, voldrem afegir-ne més en el futur, i la integració amb la resta del BernatLab (via MQTT) és natural.

25.6 Quan el P2P és millor

Dit això, el P2P té el seu espai. Algunes situacions on és millor:

- **Projectes puntuals d'un sol node:** per exemple, un sensor de pluja aïllat que només volem monitorar, sense la complexitat de TTN.
- **Educació i aprenentatge:** per entendre LoRa a fons, res millor que un P2P on veiem exactament què passa.

- **Xarxes molt privades:** si volem una xarxa totalment aïllada, sense cap servidor extern, P2P és l'única opció.
- **Hackatges ràpids:** per a un prototip d'un dia, és més ràpid fer P2P que configurar LoRaWAN.

25.7 LoRaWAN: la pila de protocols completa

Quan parlem de LoRaWAN, tenim quatre capes:

1. **Capa física (PHY):** la modulació LoRa (Semtech).
2. **Capa MAC:** el protocol LoRaWAN (LoRa Alliance).
3. **Capa de xarxa:** el network server (TTN, ChirpStack, Lloriot).
4. **Capa d'aplicació:** el application server (pot ser el mateix network server, o un servei extern).

A més, hi ha components opcionals:

- **Servidor de join:** gestiona l'autenticació inicial dels nodes.
- **Servidor de dispositius:** emmagatzema l'estat i les claus de cada node.
- **Servidor d'aplicació:** processa les dades de cada aplicació.

TTN integra tot això en una sola consola. ChirpStack els separa en serveis independents, més modular però més complex.

25.8 El paper de cada peça

Node (End Device)

El node és el sensor. Té un microcontrolador (ESP32, Pico, etc.) i un mòdul ràdio LoRa (SX1262, SX1276, etc.). Executa la pila LoRaWAN (LMIC, per exemple), gestiona les claus, i transmet.

Gateway

El gateway és el "traductor" entre el món LoRa (ràdio) i el món IP (Internet). Té un mòdul concentrador (SX1302 o SX1303, no pas SX1262 — el SX1302 pot rebre 8 canals simultànies) i una connexió a Internet. La seva feina és: escoltar totes les freqüències a 868 MHz, detectar transmissions LoRa, i enviar-les al network server.

Quan un gateway rep un missatge, l'envia al network server amb la informació de:

- Freqüència, SF, BW del senyal rebut.
- RSSI, SNR.
- Timestamp (per calcular temps de vol i triangular).
- Payload xifrat (el gateway no el pot desxifrar; només el redirigeix).

Network Server

El network server és el cervell. Funcions:

- Autenticar cada node (verificant DevEUI, AppKey).

- Desxifrar el payload (usant les claus de sessió derivades).
- Desduplicar missatges que arriben de múltiples gateways.
- Gestionar l'ADR.
- Encaminar el payload a l'application server correcte.
- Gestionar downlinks (respostes del node).

Application Server

L'application server és on aterren les dades ja desxifrades. Pot ser:

- La mateixa consola web de TTN (amb UI per visualitzar).
- Un servei MQTT (TTN publica els payloads a un broker MQTT, i nosaltres ens hi connectem).
- Un webhook HTTP (TTN fa una petició POST a una URL nostra).
- Un servei d'integració (AWS IoT, Azure IoT Hub, etc.).

Al BernatLab, farem servir **integració MQTT**: TTN publicarà cada missatge rebut a un broker MQTT, i la resta de la cadena (Telegraf, InfluxDB, Node-RED, Grafana) consumirà des d'allà.

25.9 Com triar el hardware

Per al gateway

Tres opcions principals:

1. **Waveshare SX1302 LoRaWAN Gateway HAT** (~50-80 €): un mòdul HAT per a Raspberry Pi, amb el chip SX1302. És la opció més comuna per a homelabs. Muntura fàcilment, bona documentació.
1. **RAK2287** (~70-100 €): un mòdul USB o mini-PCIe, basat en SX1302. Compatible amb moltes plaques (Raspberry, NUC, etc.).
1. **Dragino LPS8** (~200-300 €): un gateway comercial complet, amb caixa, PoE, antena externa. Més robust però més car.
1. **Construir amb un SX1302 + Raspberry Pi**: opció DIY, requereix soldar i configurar, però la més barata.

Per a Hort Osona, recomano la **opció 1 (Waveshare SX1302 HAT)** per la relació preu/facilitat.

Per als nodes

Tres famílies principals:

1. **ESP32 + SX1262 breakout**: màxima flexibilitat, però cal muntar a protoboard o PCB.
2. **Heltec LoRa 32 V3**: ESP32 + SX1262 integrats, amb pantalla OLED, antena, bateria. Molt còmode per a prototips. (~25 €)
3. **TTGO LoRa**: similar a Heltec, una mica més cara però bona qualitat. (~30 €)

4. **RAK3172**: només el mòdul (no placa de desenvolupament), pensat per a productes comercials. (~15 €)

5. **LILYGO T-Beam**: ESP32 + SX1262 + GPS. Per a nodes mòbils o geolocalització. (~40 €)

Per a Hort Osona, recomano la **Heltec LoRa 32 V3** per començar: tot integrat, fàcil de programar, bona documentació, i barata.

25.10 Programari: el que instal·larem

A la Raspberry (gateway):

- **Concentratord** (programari de gateway, abans lora-packet-forwarder).
- **Packet forwarder** (processat de paquets, configurable).

A l'ordinador o Raspberry (recepció):

- **Mosquitto** (broker MQTT, ja instal·lat al M2).
- **Telegraf** (recol·lector, ja instal·lat al M2).
- **InfluxDB** (base de dades, ja instal·lat al M2).
- **Node-RED** (processament, ja instal·lat al M2).
- **Grafana** (visualització, ja instal·lat al M2).

Al node:

- **Arduino + MCCI LoRaWAN LMIC** o **RadioLib**.
- O **MicroPython + RadioLib** o una implementació específica.

Al network server (TTN, allotjat al núvol):

- Consola web de TTN.
- Integració MQTT.

25.11 Estimació de cost total

Per a un node LoRaWAN + gateway a Hort Osona, amb 1 node inicial i 1 gateway propi:

Component	Preu (EUR)	Notes
Raspberry Pi 4 (4GB)	60	Ja la tenim.
Waveshare SX1302 HAT	70	Gateway LoRaWAN.
Antena 868 MHz (gateway)	15	Externa, amb cable.
Heltec LoRa 32 V3	25	Node.
Antena 868 MHz (node)	5	Sol soldar a la Heltec.
Sensor BME280	5	T, H, P.
Placa solar + bateria	25	Per a ús autònom.

Component	Preu (EUR)	Notes
Caixa estanca	10	Per a ús exterior.
Cables, connector, etc.	10	
Total	**~225 €**	Per a 1 node + 1 gateway.

Per afegir nodes addicionals: ~50-60 € per node (Heltec + antena + sensor).

25.12 Resum

En aquest capítol hem vist les dues grans topologies per a xarxes LoRa: **LoRaWAN** (amb gateway i network server) i **LoRa P2P** (directe entre dos mòduls). Hem après els avantatges i inconvenients de cada una, hem decidit que per a Hort Osona la millor opció és **LoRaWAN amb TTN i un gateway propi** a la Raspberry, hem vist quin hardware cal, i hem estimat el cost. En el proper capítol ens endinsarem a la capa LoRaWAN: TTN, device profiles, formats de payload, i decoders.

25.13 Exercicis pràctics

1. Crea un compte a The Things Network: <https://console.thingsnetwork.org/>
2. Crea una aplicació nova a la consola de TTN. Dona-li un nom identificatiu (per exemple, `hort-osona-bernat`).
3. Afegeix un dispositiu de prova a l'aplicació. Anota el DevEUI, AppEUI, i AppKey generats.
4. Mira la documentació de TTN sobre data formats: <https://www.thoughts.com/en/docs/concepts/data-formats/>
5. Comprova quin data rate (DR0-DR5) t'ofereix TTN a la teva regió.
6. Fes una taula amb el cost total estimat del teu projecte LoRa, segons els components que vulguis comprar.
7. Documenta al README del projecte quina decisió has pres: LoRaWAN, P2P, o un híbrid.

Paraules clau: **LoRaWAN, LoRa, P2P, gateway, network server, application server, TTN, The Things Stack, ChirpStack, DevEUI, AppEUI, AppKey, OTAA, ABP, classe A, classe B, classe C, ADR, downlink, uplink, payload, decoder, codec, CayenneLPP, JSON, MQTT, Telegraf, InfluxDB, Node-RED, Grafana, Concentrator, packet forwarder, SX1302, SX1303, SX1262, ESP32, Heltec, TTGO, RAK, Waveshare, hardware, cost, comparativa, avantatges, inconvenients, decisió, node, end device, EU868, EU433, US915, 868 MHz, 915 MHz, ISM, duty cycle, ETSI, normativa, LoRa Alliance, Semtech, codi obert, integració, broker, application server, ChirpStack Gateway Bridge, MQTT broker, broker extern, broker intern, seguretat, AES-128, xifratge, autenticació, roaming, ADR, spreading factor, SF, BW, RSSI, SNR, packet loss, link budget, EUI-64, EUI-48, IEEE 802.15.4g, EU868, EU433, US915, AS923, AU915, CN470, KR920, IN865, regions, ISM band, regional parameters, Lora Alliance, regional parameters document, TTN v3, TTN v2, TTS, The Things Stack, LNS, JOIN, OTAA, ABP, session keys, NwkSKey, AppSKey, join procedure, join request, join accept, join server, device address, DevAddr, frame counter, FCnt, FCntUp, FCntDown, FPort, FRMPayload, MAC header, MHDR, Join-Request, Join-Accept, unconfirmed data up, confirmed data up, RX1, RX2, receive windows, transmission timing, hop limit, ADR bit, ADRACKReq, ADRACKCnt, LinkADRReq,**

LinkADRAAns, DutyCycleReq, DutyCycleAns, RXParamSetupReq, RXParamSetupAns, DevStatusReq, DevStatusAns, NewChannelReq, NewChannelAns, RXTimingSetupReq, RXTimingSetupAns, class A, class B, class C, MAC commands, fopts, FOptsLen, payload size, maximum payload size, dwell time, dwell time limitation, EU868 dwell time, 868 MHz dwell time, 400 ms dwell time, 868.0 MHz, 868.6 MHz, 868.7 MHz, 869.2 MHz, 869.4 MHz, 869.65 MHz, ETSI EN 300 220, ERC Recommendation 70-03, duty cycle, 1% duty cycle, 0.1% duty cycle, 10% duty cycle, listen before talk, LBT, polite spectrum access, polite LoRa, polite LoRaWAN, polite protocol, polite stack, polite access.

Capítol 26 — La capa LoRaWAN: TTN, device profiles, payloads

"LoRaWAN és com parlar una llengua: tens un vocabulari, una gramàtica, i unes normes de cortesia. Un cop les domines, parlar amb un node és natural."

26.1 Què és The Things Network (TTN)

The Things Network (ara **The Things Stack**, TTS) és un **network server LoRaWAN comunitari i gratuït**, allotjat al núvol per la fundació The Things Network. Va començar el 2015 com un projecte de crowdsourcing per cobrir el món amb gateways comunitaris, i actualment té cobertura a centenars de països, inclosa una bona part de Catalunya i la resta d'Espanya.

La URL de la consola web és <https://console.thingsnetwork.org/>. Allà podrem:

- Crear **aplicacions** (agrupacions de dispositius).
- Registrar **dispositius** (nodes).
- Configurar **integracions** (MQTT, webhooks, etc.).
- Veure les **trames rebudes** en temps real.
- Monitorar la salut dels **gateways** propers.

TTN és la millor opció per començar amb LoRaWAN perquè:

- És **gratuït** per a ús no comercial, amb un volum raonable.
- **No requereix instal·lar cap servidor** propi.
- Té una **documentació excel·lent**.
- **Comunitat gran** amb milers de projectes, fòrums actius, tutorials.
- **Integracions fàcils** amb MQTT, webhooks, AWS, Azure, etc.

26.2 Crear un compte i una aplicació

El primer pas és crear un compte a la consola de TTN. Només cal un correu electrònic. Un cop registrats:

1. **Creem una aplicació** amb un nom identificatiu. Per a Hort Osona, jo faria servir `hort-osona-bernat`.
2. **Anotem l'Application ID i l'Application EUI** (un identificador únic generat per TTN).
3. **Accedim a la secció "Integrations"** per afegir la integració MQTT.

La consola web de TTN és intuïtiva. Si tens dubtes, la documentació oficial és a <https://www.thoughts.com/en/docs/>.

26.3 DevEUI, AppEUI, AppKey: els identificadors

Cada node LoRaWAN té **tres identificadors** principals:

- **DevEUI**: identificador únic del dispositiu, similar a una adreça MAC. 64 bits. Cada node ha de tenir-ne un d'únic.
- **AppEUI** (o **JoinEUI**): identificador de l'aplicació. 64 bits. Indica a quin "tenant" pertany el node.
- **AppKey**: clau mestra de 128 bits, usada per autenticar el node durant el procés de join.

A TTN, quan registrem un node, podem:

- **Generar automàticament** els identificadors (TTN ens els dona).
- **Introduir-los manualment** si el node ja ve amb identificadors preconfigurats (com passa amb molts mòduls comercials).

Els valors generats per TTN es mostren **una sola vegada**. Cal guardar-los en un lloc segur (per exemple, un fitxer `.env` o un document xifrat).

26.4 OTAA vs ABP: els dos mètodes d'unió

Quan un node vol connectar-se a la xarxa LoRaWAN, ho pot fer de dues maneres:

OTAA (Over-The-Air Activation)

Aquest és el **mètode recomanat** i l'estàndard de facto. Funciona així:

1. El node envia un **Join Request** amb el seu DevEUI, AppEUI, i un nonce.
2. El network server valida la sol·licitud usant l'AppKey.
3. Si tot és correcte, el NS envia un **Join Accept** amb un DevAddr (adreça dinàmica) i les claus de sessió (NwkSKey, AppSKey).
4. A partir d'aquí, el node pot transmetre amb les claus de sessió.

Avantatges d'OTAA:

- Més segur: les claus de sessió es renegocien cada vegada.
- Permet el roaming: el node pot canviar de network server.
- Permet l'ADR: el NS pot ajustar els paràmetres.

ABP (Activation By Personalization)

En aquest mètode, les claus de sessió (NwkSKey, AppSKey) i el DevAddr es configuren **directament al node**, sense cap handshake. El node ja està "unida" a la xarxa per sempre, amb aquestes claus fixes.

Inconvenients d'ABP:

- Si les claus es comprometen, cal reprogramar el node.
- No suporta roaming.
- No suporta ADR de manera nativa.

Recomanació: usa **OTAA sempre**, tret que tinguis un motiu específic per usar ABP.

26.5 Device profiles

TTN permet definir **device profiles** que descriuen les capacitats d'un tipus de dispositiu:

- Versió de LoRaWAN (1.0.2, 1.0.3, 1.0.4, 1.1).
- Versió de la Regional Parameters (RP001-1.0.3, etc.).
- Capacitat de l'ADR.
- Classes suportades (A, B, C).
- Màxima potència de transmissió.
- Fabrique i model.

Els device profiles ajuden a mantenir la consistència quan tenim molts dispositius similars. Per a Hort Osona, podem crear un device profile comú per a tots els nodes de sensors ambientals.

26.6 Payload formats: CayenneLPP i altres

Quan un node LoRaWAN transmet, el payload és una seqüència de bytes (típicament 5-50 bytes). Com interpretem aquests bytes?

Hi ha diversos estàndards i convencions:

CayenneLPP (Cayenne Low Power Payload)

És el format més popular per a sensors LoRaWAN. Creat per myDevices, està pensat per ser molt eficient en bytes. Cada dada es codifica amb:

- 1 byte: tipus de dada (per exemple, 0x67 = temperatura).
- 1 byte: canal (per exemple, 0x01 = canal 1).
- N bytes: el valor (per exemple, 2 bytes per a un enter amb signe de 16 bits, o 4 bytes per a un float).

Exemples de canals CayenneLPP:

Tipus	Valor	Bytes
0x67 (Temperatura)	graus C × 10 (signed 16-bit)	2
0x68 (Humitat)	% × 2 (unsigned 16-bit)	2
0x69 (Acceleròmetre)	tres eixos (3 × signed 16-bit)	6
0x6E (Il·luminació)	lux (unsigned 16-bit)	2
0x75 (Humitat del sòl)	% (unsigned 8-bit)	1
0x02 (Sortida analògica)	valor (float, 4 bytes)	4
0x03 (Sortida analògica)	valor (float, 4 bytes)	4

Exemple de payload CayenneLPP per a un sensor amb temperatura, humitat i lluminositat:

67 01 00 FF # Temperatura al canal 1: 0x00FF = 255, /10 = 25.5°C

```
68 02 00 B6    # Humitat al canal 2: 0x00B6 = 182, /2 = 91%
6E 03 0B B8    # Lluminositat al canal 3: 0x0BB8 = 3000 lux
```

Total: 12 bytes. Molt eficient.

JSON sobre LoRaWAN

Una altra opció és enviar JSON com a payload:

```
{"t": 25.5, "h": 91, "l": 3000, "id": "n1", "v": 1}
```

Avantatges: llegible, fàcil de debugar. Inconvenients: ocupa més bytes. Un missatge de 50 bytes amb JSON pot trigar molt més a transmetre que un CayenneLPP equivalent.

Binari personalitzat

Si tenim moltes dades idèntiques, podem definir el nostre propi format binari. Per exemple, 2 bytes per a la temperatura, 1 byte per a la humitat, 2 bytes per al voltatge de la bateria. Total: 5 bytes per a tres dades. Molt eficient, però menys estàndard.

26.7 Payload formatters a TTN

TTN permet definir **payload formatters** que interpreten el payload del node:

- **Decoder**: converteix el payload binari en JSON per a l'aplicació.
- **Converter** (uplink): format JSON que TTN rep i publica.
- **Encoder** (downlink): converteix JSON en payload binari per enviar al node.

Exemple de decoder CayenneLPP en JavaScript:

```
function decodeUplink(input) {
  var decoded = {};
  var i = 0;
  while (i < input.bytes.length) {
    var channel = input.bytes[i++];
    var type = input.bytes[i++];
    switch (type) {
      case 0x67: // Temperatura
        var t = (input.bytes[i++] << 8) | input.bytes[i++];
        if (t > 0x7FFF) t -= 0x10000;
        decoded.temperatura = t / 10.0;
        break;
      case 0x68: // Humitat
        var h = (input.bytes[i++] << 8) | input.bytes[i++];
        decoded.humitat = h / 2.0;
        break;
      // Afegir altres tipus segons calgui
    }
  }
  return {
    data: decoded,
    warnings: [],
    errors: []
  };
}
```

Aquest codi es pot enganxar directament a la consola de TTN, secció "Payload formatters" del dispositiu.

26.8 Integració amb el broker MQTT

La integració clau per al BernatLab és **MQTT**. TTN ens permet configurar una integració MQTT on cada vegada que un node transmet, el payload desxifrat (en JSON) es publica a un broker MQTT.

A TTN, a la secció "Integrations":

1. **Triem "MQTT"**.
2. **Configurem el broker:** - **Broker Address:** `mqtt://100.115.134.76:1883` (la Tailscale IP del BernatLab). - **Username:** `ttn-bridge` (o el que hàgim creat a Mosquitto). - **Password:** la contrasenya.
3. **Triem el topic prefix** (per defecte, `v3/{application_id}@{tenant_id}/devices/{device_id}/`).
4. **Desem.**

Un cop configurat, cada vegada que un node transmet, TTN publicarà un missatge JSON al broker MQTT. Aquest missatge el pot consumir Telegraf (per guardar-lo a InfluxDB) o Node-RED (per processar-lo).

Format del missatge MQTT

El missatge que TTN publica té aquesta estructura:

```
{
  "end_device_ids": {
    "device_id": "eui-70b3d57ed004f1ce",
    "application_ids": {
      "application_id": "hort-osona-bernat"
    },
    "dev_eui": "70B3D57ED004F1CE",
    "join_eui": "0000000000000000"
  },
  "received_at": "2026-07-08T12:34:56Z",
  "uplink_message": {
    "f_port": 2,
    "f_cnt": 42,
    "frm_payload": "GAEABbYBmAu4",
    "decoded_payload": {
      "temperatura": 25.5,
      "humitat": 91,
      "lux": 3000
    },
    "rx_metadata": [{
      "gateway_ids": {
        "gateway_id": "raspi-gw-1"
      },
      "rssi": -67,
      "snr": 9.5,
      "channel_rssi": -67,
      "channel_index": 5
    }],
    "settings": {
      "data_rate": {
        "lorawan": {
          "bandwidth": 125000,
          "spreading_factor": 9
        }
      }
    }
  },
}
```

```

    "frequency": "868100000"
  }
}
}

```

Com veieu, el payload decodificat ja ve en JSON, juntament amb metadades útils: RSSI, SNR, data rate, etc.

Topics MQTT

Per defecte, TTN publica a:

```
v3/{application_id}@{tenant_id}/devices/{device_id}/up
```

on **up** indica que és un missatge uplinki (del node al servidor). Els altres tipus de missatge són:

- **up**: uplink del node.
- **down**: downlink (resposta del servidor al node).
- **join**: peticions de join (OTAA).
- **ack**: confirmacions.
- **error**: errors.

26.9 Downlink: enviar comandes al node

A més de rebre dades dels nodes, podem **enviar ordres** (downlinks). Això és útil per:

- Canviar la configuració d'un node a distància.
- Activar un actuator (per exemple, obrir una vàlvula de reg).
- Fer un "ping" per comprovar si el node està viu.

Limitacions del downlink:

- **Només a classe A**: el node només pot rebre durant les finestres RX1 i RX2, just després d'haver transmès.
- **Duty cycle igual**: cada downlink compta contra el duty cycle.
- **FPort**: el downlink s'envia en un port concret (1-223).

A TTN, podem programar un downlink des de la consola web, o via API, o escoltant un topic MQTT específic.

26.10 Classes de dispositius

LoRaWAN defineix tres classes:

- **Classe A**: el node pot rebre **només immediatament** després d'haver transmès. La majoria de sensors són classe A per estalviar energia.
- **Classe B**: el node pot rebre en finestres programades (pings del network server). Sincronitza amb el NS periòdicament.

- **Classe C:** el node pot rebre **continuant** (excepte quan està transmetent). Consumeix més energia, però permet downlinks immediats. Útil per a actuadors o nodes endollats.

Per a sensors amb piles, **classe A** és l'estàndard. La Heltec LoRa 32 V3 pot treballar en classe A o C.

26.11 Procediment de join: detall tècnic

Quan un node OTAA arrenca, segueix aquest procediment:

1. **Genera un nonce aleatori** (DevNonce).
2. **Construeix el Join Request** amb DevEUI, AppEUI, DevNonce.
3. **Xifra el Join Request** amb l'AppKey.
4. **Envia el Join Request** a un canal aleatori.
5. **Escolta el Join Accept** a RX1 (al cap de 1 segon $\pm 20 \mu s$ al canal de resposta) o RX2 (al cap de 2 segons al canal i data rate configurats).
6. Si rep el Join Accept, **deriva les claus de sessió** (NwkSKey, AppSKey) i el DevAddr.
7. A partir d'aquí, **pot transmetre dades normals** amb aquestes claus.

Si no rep el Join Accept, torna a provar amb un altre canal i un altre DevNonce. Després de N intents, espera més estona (backoff exponencial).

26.12 El paper de l'AppKey

L'AppKey és la clau mestra. Està emmagatzemada al node (en memòria no volàtil) i al network server (a la configuració del dispositiu). S'usa per:

- **Xifrar el Join Request** (per demostrar que el node és legítim).
- **Derivar les claus de sessió** durant el join.
- **No s'usa mai per xifrar les dades normals:** això ho fan les claus de sessió derivades.

Si l'AppKey es compromet, algú podria suplantar el node. Per tant, **cal guardar-la en lloc segur** (fitxer xifrat, gestors de secrets, etc.).

26.13 Configurar la integració MQTT a TTN

Pas a pas, en detall:

1. **Crear un usuari MQTT a Mosquitto:**

```
docker exec -it mosquitto mosquitto_passwd -c /mosquitto/config/passwordfile ttn-bridge
```

(Per afegir a un fitxer existent, sense -c).

1. **Configurar les ACLs** per a l'usuari `ttn-bridge`:

```
user ttn-bridge
topic write v3/+/@/devices/+/up
topic write v3/+/@/devices/+/join
topic write v3/+/@/devices/+/down
```



```
topic write v3/+/@/devices/+/ack
topic read v3/+/@/devices/+/down
```

1. **A la consola de TTN**, a la secció "Integrations":

- Clic a "MQTT".
- Clic a "Add integration".
- A la URL del broker: `mqtt://100.115.134.76:1883`.
- Username: `ttn-bridge`.
- Password: la contrasenya.
- Desar.

1. **Verificar** que TTN es pot connectar al broker. A la consola, a la secció "Integrations", hauria d'aparèixer "Connected".

26.14 El paper de l'Application Server

Al BernatLab, l'**application server** és el que consumeix les dades de TTN. En el model de TTN, això podem ser:

- La **consola web de TTN** (per visualitzar i debugar).
- Un **broker MQTT** (com el nostre Mosquitto).
- Un **servei extern** (AWS IoT, Azure IoT Hub, etc.).

Per al nostre cas, l'application server és el **broker Mosquitto del BernatLab**, i els consumidors són Telegraf, Node-RED, i Grafana.

26.15 Resum

En aquest capítol hem après com funciona la capa LoRaWAN: els identificadors (DevEUI, AppEUI, AppKey), els mètodes d'unió (OTAA recomanat, ABP com a excepció), els formats de payload (CayenneLPP, JSON, binari personalitzat), la integració amb MQTT, els downlinks, les classes de dispositius, i el procediment de join. Hem configurat la integració MQTT a TTN perquè les dades dels nodes arribin al broker del BernatLab. En el proper capítol muntarem un gateway LoRaWAN a la Raspberry amb Concentratord.

26.16 Exercicis pràctics

1. Crea un compte a TTN: <https://console.thingsnetwork.org/>
2. Crea una aplicació anomenada `hort-osona-bernat`.
3. Afegeix un dispositiu de prova a l'aplicació. Anota DevEUI, AppEUI, AppKey.
4. Configura la integració MQTT a TTN apuntant al broker del BernatLab.
5. Prova de subscriure't a `v3/#` al broker Mosquitto. Hauries de veure algun missatge de keepalive.
6. Escriu un payload formatter (decoder CayenneLPP) per a un node que envia temperatura, humitat, i humitat del sòl.

7. Documenta al README del projecte els identificadors de cada node, mai en text pla al repositori.

Comandes útils:

```
# Crear usuari MQTT per a TTN
docker exec mosquitto mosquitto_passwd -c /mosquitto/config/passwordfile ttn-bridge
```

```
# Provar la connexió
mosquitto_sub -h 100.115.134.76 -t "v3/#" -v -u bernat -P CONTRASENYA
```

Paraules clau: **TTN, The Things Network, The Things Stack, TTS, consola, aplicació, dispositiu, DevEUI, AppEUI, AppKey, NwkSKey, AppSKey, DevAddr, OTAA, ABP, join, join request, join accept, device profile, payload, CayenneLPP, JSON, binari, decoder, encoder, formatter, MQTT, integració, broker, topic, uplink, downlink, classe A, classe B, classe C, RX1, RX2, duty cycle, dev nonce, app nonce, fcnt, fport, EUI-64, EUI-48, IEEE 802.15.4g, xarxa, network server, application server, MQTT broker, Mosquitto, ACL, usuari, contrasenya, keepalive, packet forwarder, concentrator, xifrat, AES-128, integritat, autenticació, anti-replay, FCntUp, FCntDown, session keys, session context, NwkSKey, AppSKey, derivació, seguretat LoRaWAN, EU868, 868 MHz, EU868, channels, data rates, DR0-DR5, ADR, datr, payload size, maximum payload size, dwell time, ETSI, ISM, sub-banda, normativa.**

Capítol 27 — Gateway LoRaWAN a la Raspberry amb Concentratord

"Un gateway LoRaWAN és com una torre de telefonia, però minúscula: 50 grams, 50 euros, i pot cobrir quilòmetres. La diferència és que el muntas tu."

27.1 Què és un gateway LoRaWAN

Un **gateway** LoRaWAN és un dispositiu que escolta les transmissions LoRa a 868 MHz i les redirigeix al network server (com The Things Stack) per Internet. A diferència d'un node (que és un sensor amb poca potència), un gateway ha de ser capaç de rebre **molts canals simultàniament** i durant **tot el temps**.

Per fer-ho, els gateways moderns usen un xip **concentrador** que pot rebre 8 o més canals alhora. El més popular actualment és el **SX1302** (o el seu successor, el **SX1303**), de Semtech. Aquest xip no és un transmissor LoRa normal (com el SX1262); és un processador de senyals dedicat que pot rebre transmissions de múltiples nodes alhora.

El gateway típic té:

- Un **mòdul concentrador** (SX1302).
- Un **microcontrolador** o **ordinador** que gestiona el mòdul i la connexió a Internet.
- Una **antena externa** (per a bon abast).
- Un **programari** (packet forwarder) que envia els paquets rebuts al network server.

27.2 Tipus de gateways

Hi ha diverses maneres de tenir un gateway LoRaWAN:

Gateways comercials

Aparells complets, amb caixa, antena, i software preinstal·lat:

- **Dragino LPS8**: molt popular, robust, amb PoE. ~200-300 €.
- **RAK7268 / RAK7258**: bons, amb Wi-Fi i 4G. ~250-350 €.
- **MikroTik LoRa**: barat però limitat. ~100 €.

DIY amb Raspberry Pi + mòdul SX1302

La opció més comuna per a homelabs. Es compon de:

- Una **Raspberry Pi** (que ja tenim al BernatLab).
- Un **mòdul SX1302** connectat per **SPI** o **USB**.
- **Software**: Concentratord (o altres com ChirpStack Gateway Bridge).

Aquesta opció és la que veurem en detall, perquè és la que farem servir al BernatLab.

Gateways virtuals

En alguns casos, podem usar un gateway d'un altre (per exemple, un gateway TTN comunitari) sense tenir-ne un de propi. Això és el que passa si hi ha cobertura TTN a menys de 5-10 km amb visió directa.

27.3 Hardware: el mòdul SX1302

Hi ha diversos mòduls al mercat:

1. **Waveshare SX1302 LoRaWAN Gateway HAT**: la opció més popular. Es connecta directament als pins GPIO de la Raspberry Pi. ~70 €.
1. **RAK2287**: mòdul mini-PCIe amb SX1302. Es connecta a una placa base com la RAK2287 PCIe. ~80-100 €.
1. **Seeed Studio WM1302**: mòdul mini-PCIe similar al RAK. ~70 €.
1. **SX1302 + Raspberry directament**: mòdul sol + cable, per als més agosserats. ~50 €.

Al BernatLab, recomanem el **Waveshare SX1302 HAT** per la facilitat d'instal·lació. Només cal encaixar-lo als pins GPIO de la Raspberry, sense cables, sense soldar.

Característiques del SX1302

- **8 canals de recepció simultània** (pot rebre transmissions de fins a 8 nodes alhora).
- **SF7-SF12** suportat.
- **125 / 250 / 500 kHz** de BW.
- **868 / 915 / 923 MHz** (multibanda).
- **Connexió SPI** (a través dels pins GPIO de la Raspberry).
- **Consum**: ~1-2 W en recepció.

27.4 Antena

L'antena és **tan important com el mòdul**. Una mala antena pot reduir l'abast a la meitat o més.

Per a 868 MHz, opcions:

- **Antena de rubber ducky** (petita, omnidireccional, ~5 dBi): bona per a provar.
- **Antena de fibra de vidre** (més llarga, ~5-8 dBi): millor rendiment, bona per a exteriors.
- **Antena direccional Yagi** (~8-12 dBi): molt abast en una direcció concreta, útil si el node està en una posició fixa.

A 868 MHz, una antena de 1/4 d'ona fa ~8.6 cm. Una de 1/2 d'ona, ~17 cm. Una de 5/8 d'ona, ~35 cm.

Al BernatLab, recomanem una **antena de fibra de vidre omnidireccional** per a ús general. Si tenim una posició fixa del node, podem considerar una Yagi per augmentar l'abast.

Cable i connectors

El SX1302 HAT de Waveshare porta un **connector U.FL** (IPEX) a la placa. Per connectar una antena externa, cal un **pigtail U.FL a SMA** (un cable petit d'uns 10-15 cm). Un extrem va al HAT, l'altre a l'antena.

Compte amb els cables massa llargs: a 868 MHz, cada metre de cable perd ~0.5-1 dB. Mantenim el cable el més curt possible.

27.5 Software: Concentratord

Concentratord (abans conegut com a **lora-packet-forwarder**) és el programari estàndard per a gateways basats en SX1302. Està desenvolupat per xdegaye (Benjamin)} i mantingut activament.

Hi ha dues variants:

- **Concentratord**: la nova versió, refeta des de zero, modular.
- **lora-packet-forwarder**: la versió clàssica, encara funcional però menys activa.

Recomanem **Concentratord** per a noves instal·lacions.

Altres opcions de programari

- **ChirpStack Gateway Bridge**: si volem integrar el gateway amb un ChirpStack autoallotjat.
- **TTN Packet Forwarder**: oficial de TTN, però menys flexible.

27.6 Instal·lació de Concentratord a la Raspberry

Al BernatLab, la Raspberry ja té Docker. Podem executar Concentratord com un contenidor, o com a servei natiu. Recomanem el **contenidor Docker** per la facilitat de gestió.

Opció 1: Contenidor Docker (recomanada)

Al `docker-compose.yml` del BernatLab, afegim un servei:

```
services:
  concentratord:
    image: chirpstack/chirpstack-concentratord:2
    container_name: concentratord
    restart: unless-stopped
    devices:
      - /dev/spidev0.0:/dev/spidev0.0
    volumes:
      - ./concentratord:/etc/chirpstack-concentratord
    ports:
      - "3001:3001"
```

Això munta el dispositiu SPI de la Raspberry al contenidor (perquè pugui parlar amb el SX1302) i munta una carpeta amb la configuració.

Opció 2: Instal·lació nativa

```
# Afegir el repositori
sudo apt install -y software-properties-common
sudo apt-key adv --keyserver keyserver.ubuntu.com --recv-keys 1FF3F2DB
sudo add-apt-repository ppa:chirpstack/chirpstack-gateway
sudo apt update
sudo apt install chirpstack-concentrator
```

27.7 Configuració de Concentrator

El fitxer principal és `chirpstack-concentrator.toml`. A `/home/bernat/homelab/concentrator/`:

```
# Configuració del gateway
[gateway]
# Identificador únic del gateway (EUI-64)
gateway_id = "AA555A0000000001"

# Servidor TTN
server_address = "eu1.cloud.thethings.network:1700"

# Interval en què enviem estadístiques
statistics_interval = "30s"

# Ubicació del gateway (per a TTN)
latitude = 41.9304
longitude = 2.2546
altitude = 500

# Configuració del concentrador
[concentrator]
# El SX1302 es comunica amb la Raspberry per SPI
spi_path = "/dev/spidev0.0"
spi_speed_hz = 2000000
```

Gateway ID: el EUI-64

Cada gateway ha de tenir un **EUI-64** únic. Podem generar-lo:

```
echo "AA555A$(openssl rand -hex 5 | tr a-f A-F | cut -c 1-10)"
```

O simplement fem servir un valor vàlid que no coincideixi amb cap altre gateway de TTN. El prefix `AA555A` és un prefix reservat per a gateways experimentals.

Configurar el servidor TTN

El `server_address` ha de ser l'endpoint de TTN. Per a la UE, és `eu1.cloud.thethings.network:1700`. Per a altres regions, TTN té altres servidors (`nam1.cloud.thethings.network`, `au1.cloud.thethings.network`, etc.).

27.8 Registrar el gateway a TTN

Un cop tenim el gateway funcionant, cal registrar-lo a TTN:

1. A la consola de TTN, anem a la secció **Gateways**.
2. Clic a **+ Register gateway**.
3. **Gateway EUI**: introduïm el EUI-64 que hem configurat a `chirpstack-concentrator.toml`.

4. **Gateway ID:** un nom únic (per exemple, `raspi-hortosona-gw`).
5. **Gateway name:** un nom descriptiu.
6. **Frequency plan:** `Europe 868 MHz` (o el que correspongui).
7. **Router:** el que ens suggereixi TTN (no importa gaire per a un homelab).
8. **Desem.**

TTN ens donarà un **Gateway Server Address** (que hem de posar al `server_address` de la nostra configuració) i una clau d'autenticació. A Concentrator 2.x, l'autenticació és per **API key** o per **TLS client certificates**, configurable al `chirpstack-concentrator.toml`.

Exemple de configuració amb API key

```
[gateway]
gateway_id = "AA555A0000000001"
server_address = "eu1.cloud.thethings.network:1700"

[gateway.auth]
api_key = "NNSXS.XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX..."
```

27.9 Verificar la connexió

Un cop en marxa, podem comprovar:

1. **A la consola de TTN**, a la pàgina del gateway, veure l'estat. Si apareix "Connected" o "Last seen: recent", tot va bé.
2. **Els logs del contenidor:**

```
docker compose logs -f concentrator
```

1. **L'estat del SX1302** amb la comanda `chirpstack-concentrator-ctl`:

```
docker exec concentrator chirpstack-concentrator-ctl gateway status
```

27.10 Proves amb un node

Per validar que el gateway funciona, podem fer servir un node comercial o un prototip:

1. **Configurar un node** (per exemple, una Heltec LoRa 32 V3) amb el nostre DevEUI i AppEUI.
2. **Programar-lo** per transmetre cada 30 segons.
3. **Mirar la consola de TTN:** a la secció "Live data" o a la pàgina del node, hauríem de veure els uplink.

Si veiem els uplink, tenim:

- Gateway LoRaWAN funcionant.
- Concentrator connectat a TTN.
- Node transmettent correctament.
- Xarxa TTN rebent i desxifrant.

Si NO veiem res, caldrà depurar. Aquest és el tema del capítol 32.

27.11 Configuració avançada

Múltiples concentradors (no típic)

Si volem tenir dos SX1302 a la mateixa Raspberry (per diversitat d'antena, per exemple), cal configurar dos serveis `concentrator` amb pins SPI diferents.

Múltiples xarxes (TTN + ChirpStack)

Concentrator 2.x pot enviar els mateixos uplink a múltiples network servers. Cal afegir múltiples seccions `[gateway]` o usar la funció de "multi-server".

Filtratge de canals

Podem configurar quins canals de 868 MHz volem rebre. Per defecte, els 8 canals principals estan actius. Podem desactivar-ne alguns per reduir la càrrega de la CPU.

GPS del gateway

Si la Raspberry té un mòdul GPS connectat, podem passar la posició exacta a TTN. Això millora la cobertura i permet triangular.

27.12 Monitoratge del gateway

Com monitorem el gateway?

Uptime Kuma

Afegim un monitor al port 3001 (el port per defecte de la interfície de debug de Concentrator):

- **Tipus:** HTTP(s).
- **URL:** `http://100.115.134.76:3001/`.
- **Interval:** 60 segons.

Alternativament, podem monitorar si el procés està viu:

```
docker ps | grep concentrator
```

Logs a Uptime Kuma

Si volem centralitzar els logs, podem muntar un volum compartit i configurar logrotate. Però això és opcional per a un homelab.

Estadístiques a TTN

TTN ensenya estadístiques del gateway: nombre d'uplinks rebuts, RSSI mitjà, SNR mitjà, taxa d'error. Això és molt útil per diagnosticar.

27.13 Consum d'energia i alimentació

El SX1302 HAT consumeix ~1-2 W. La Raspberry Pi 4 consumeix ~3-7 W segons la càrrega. Total, ~5-9 W.

Per a ús 24/7, podem:

- Usar l'alimentador oficial de la Raspberry (5.1V, 3A).
- Considerar PoE (Power over Ethernet) si tenim un switch compatible.
- Considerar una font UPS per a talls de llum.

A llarg termini, una UPS petita (per exemple, un PowerBank amb pass-through) pot evitar reinicis innecessaris.

27.14 Posició del gateway

La **posició** del gateway és crítica. Un gateway ben col·locat pot cobrir 5-10 km; un mal col·locat, 100 metres.

Criteris:

- **Alçada:** com més amunt, millor. Teulada, torre, arbre alt.
- **Visió directa:** sense arbres, edificis, o muntanyes al mig.
- **L'allunyament de metalls:** estructures metàl·liques reflecteixen el senyal.
- **L'allunyament d'altres antenes:** per evitar interferències.
- **Accés a corrent elèctric i xarxa:** evidentment.

A Hort Osona, el millor lloc per al gateway seria la teulada de la casa, amb l'antena sobresortint. Si no és possible, una finestra alta o un balcó pot servir.

27.15 Resum

En aquest capítol hem vist com muntar un gateway LoRaWAN a la Raspberry amb un mòdul SX1302 (recomanem el WaveShare HAT) i el programari Concentratord. Hem après a configurar-lo, registrar-lo a TTN, i verificar la connexió. Hem vist com monitorar-lo amb Uptime Kuma i com col·locar-lo correctament. En el proper capítol veurem el hardware del node: ESP32 + SX1262, esquema elèctric, antena, i alimentació.

27.16 Exercicis pràctics

1. Compra o demana un mòdul SX1302 HAT (WaveShare, RAK2287, o similar).
2. Connecta'l a la Raspberry Pi 4 (apaga la Raspberry, encén el HAT als pins GPIO, torna a encendre).
3. Desplega Concentratord amb Docker a la Raspberry.
4. Configura `chirpstack-concentratord.toml` amb un EUI-64 únic.
5. Registra el gateway a TTN amb el EUI-64.
6. Comprova l'estat a la consola de TTN.
7. Configura un monitor d'Uptime Kuma per al gateway.
8. Documenta al README del projecte l'estructura del gateway, l'EUI-64, i la configuració.

Comandes útils:

```
# Verificar que el SX1302 és detectat
ls -la /dev/spidev*

# Veure els logs de Concentrator
docker compose logs -f concentrator

# Verificar l'estat del concentrador
docker exec concentrator chirpstack-concentrator-ctl gateway status

# Monitorar el tràfic
docker exec concentrator chirpstack-concentrator-ctl gateway config
```

Paraules clau: **gateway, concentrator, SX1302, SX1303, HAT, WAVESHARE, RAK2287, Seeed, RAK, Mini-PCIe, GPIO, SPI, spidev, packet forwarder, U.FL, IPEX, SMA, pigtail, antena, 868 MHz, omnidireccional, direccional, Yagi, fibra de vidre, dBi, gateway EU1, gateway ID, EU1-64, TTN, EU1.cloud.thethings.network, NAM1, AU1, router, autenticació, API key, TLS, certificat, server address, last seen, connected, last seen, live data, uplink, downlink, RSSI, SNR, posicionament, latitud, longitud, altitud, location, monitoratge, Uptime Kuma, Docker, contenidor, ChirpStack Gateway Bridge, lora-packet-forwarder, multi-server, filtratge de canals, GPS, PoE, UPS, alimentació, posicionament del gateway, visió directa, cobertura, abast, distància, alçada, interferències, reflexió, soroll, RSSI mitjà, SNR mitjà, taxa d'error, gateway status, gateway config, chirpstack-concentrator-ctl, SPI speed, 2 MHz, 8 MHz, EU868, 868 MHz, 868.1, 868.3, 868.5, 867.1, 867.3, 867.5, 867.7, 867.9.**

Capítol 28 — El node: ESP32 + SX1262, hardware i esquemes

"Un node LoRa és la combinació d'un cervell, uns sentits i una veu. El cervell és l'ESP32, els sentits són els sensors, i la veu és el SX1262."

28.1 Què necessita un node

Un node LoRaWAN al camp ha de tenir:

1. **Un microcontrolador** per llegir sensors i prendre decisions.
2. **Un mòdul ràdio LoRa** per transmetre les dades.
3. **Un sensor o sensors** (temperatura, humitat, sòl, etc.).
4. **Una antena** per transmetre.
5. **Alimentació** (bateria, placa solar, USB, etc.).
6. **Una caixa estanca** per a ús a l'exterior.

Això és molt semblant al que ja coneixem de qualsevol sistema encastat (sistema encastat = sistema informàtic integrat en un altre producte). La diferència és que la transmissió és per ràdio, no per Wi-Fi o Bluetooth.

28.2 Per què ESP32

L'ESP32 és la millor opció per a un node LoRa perquè:

- **Barat:** ~3-5 € una placa de desenvolupament amb Wi-Fi i Bluetooth integrats.
- **Potent:** dual-core 240 MHz, 520 KB de RAM, suficient per a la majoria de tasques.
- **Comunitat enorme:** llibreries per a gairebé tot, tutorials, fòrums.
- **Baix consum:** ~80 mA actiu, ~10 µA en deep sleep.
- **Moltes plaques amb LoRa integrat:** Heltec LoRa 32, TTGO LoRa, LILYGO T-Beam, etc.

Alternatives:

- **Raspberry Pi Pico + SX1262:** bona opció, però menys exemples LoRa.
- **STM32 + SX1262:** molt baix consum, però més complex de programar.
- **Arduino Nano + SX1262:** barat, però poca memòria per a la pila LoRaWAN.

28.3 Plaques ESP32 amb LoRa integrat

Aquestes plaques combinen ESP32 + SX1262 + antena + (sovint) pantalla OLED i connector de bateria. Són la millor opció per començar:

Heltec LoRa 32 V3

- **ESP32-S3FN8** (dual-core 240 MHz, 8 MB flash, 512 KB RAM, 384 KB ROM).
- **SX1262** LoRa transceiver.
- **Pantalla OLED** de 0,96" (128×64).
- **Connector per a bateria LiPo** (amb circuit de càrrega).
- **Antena** externa amb IPEX/U.FL connector.
- **Preu:** ~25 €.

TTGO LoRa (T-Beam, T-HiGrow, etc.)

Similar a Heltec, sovint amb GPS afegit en el cas del T-Beam. Una mica més cara (~30 €) però més opcions.

LILYGO T3S3 / T-Deck

Plaques noves amb pantalles de color, millor consum, suport per a LoRaWAN i Meshtastic.

Plaques "nues" (mòdul separat)

Si volem més flexibilitat, podem comprar:

- Un **ESP32-DevKit** estàndard (~5 €).
- Un **mòdul SX1262 breakout board** (~10 €).
- Cablejat manual (jumper wires, protoboard, o PCB).

Aquesta opció és més feina però permet personalitzar al 100 % la disposició.

28.4 Sensors

Depenent del que vulguem mesurar:

BME280

- **Mesura:** temperatura, humitat, pressió.
- **Interfície:** I2C o SPI.
- **Precisió:** ± 0.4 °C, ± 3 % HR, ± 1 hPa.
- **Preu:** ~3-5 €.
- **Ús:** un dels més comuns per a estacions meteorològiques.

SHT31

- **Mesura:** temperatura, humitat.
- **Interfície:** I2C.
- **Precisió:** ± 0.3 °C, ± 2 % HR.
- **Preu:** ~3-5 €.

DHT22 / AM2302

- **Mesura:** temperatura, humitat.
- **Interfície:** GPIO digital.
- **Precisió:** ± 0.5 °C, $\pm 2-5$ % HR.
- **Preu:** ~2-3 €.
- **Ús:** molt popular, però menys precís i menys fiable que el BME280.

Sensor d'humitat del sòl capacitiu

- **Mesura:** humitat del sòl (%).
- **Interfície:** analògica (0-3.3 V) o digital.
- **Preu:** ~1-3 €.
- **Ús:** clau per a Hort Osona.

Sensor de llum (LDR o BH1750)

- **LDR** (Light Dependent Resistor): resistència variable segons la llum. Analògica, molt barata, però poc precisa.
- **BH1750:** digital, mesura lux. Més car (~2 €) però precís.

Pluja (pluviòmetre)

Pluviòmetre tipus balancí: genera un impuls cada 0.2 mm de pluja. Es connecta a un pin d'interrupció de l'ESP32.

Pressió (si no tenim BME280)

BMP280: pressió + temperatura (sense humitat). Més barat que BME280.

28.5 Connexió SX1262 a ESP32

Si usem una placa amb tot integrat (Heltec, TTGO), les connexions ja estan fetes. Si usem un mòdul separat, cal connectar:

SX1262	ESP32	Funció
VCC	3.3V	Alimentació (compte: 3.3V, no 5V)
GND	GND	Terra
SCK	GPIO 18	SPI Clock
MISO	GPIO 19	SPI MISO
MOSI	GPIO 23	SPI MOSI
NSS / CS	GPIO 5	Chip Select

SX1262	ESP32	Funció
RST	GPIO 14	Reset
DIO1	GPIO 26	Interrupció
BUSY	GPIO 27	Busy (opcional)

Aquests pins són els valors per defecte a la majoria de llibreries. Es poden canviar per software.

28.6 Esquema elèctric

Per a una versió amb ESP32-DevKit + SX1262 breakout + BME280:

```

ESP32          SX1262
 3.3V  ----- VCC
 GND   ----- GND
 GPIO18----- SCK
 GPIO19----- MISO
 GPIO23----- MOSI
 GPIO5  ----- NSS
 GPIO14----- RST
 GPIO26----- DIO1
 GPIO27----- BUSY

```

```

ESP32          BME280
 3.3V  ----- VCC
 GND   ----- GND
 GPIO21----- SDA  (I2C Data)
 GPIO22----- SCL  (I2C Clock)

```

Això és el bàsic. Si volem afegir un sensor d'humitat del sòl, connectem el seu pin analògic (AO) a un pin ADC de l'ESP32 (per exemple, GPIO34).

28.7 Alimentació

Per a ús al camp, tenim diverses opcions:

Bateria LiPo + placa solar

- **Bateria:** 3.7V LiPo 18650 (3000-3500 mAh) o LiPo petita.
- **Placa solar:** 5-6V, 1-2W.
- **Carregador:** TP4056 o integrat a la placa (Heltec ja en porta).
- **Autonomia:** amb bones pràctiques, mesos o anys.

USB

- **Font:** carregador de mòbil, 5V 1A.
- **Ús:** interior, prototipatge, proves.

4 piles AA

- **Voltatge:** $4 \times 1.5V = 6V$ (necessitem un regulador a 3.3V o 5V).

- **Capacitat:** ~2500-3000 mAh.
- **Ús:** per a proves a l'hort a curt termini.

Bateria fixa (12V, tipus bateria de cotxe o solar)

- Per a instal·lacions permanents amb consum moderat.

Optimització de consum

Per allargar la vida de la bateria:

- **Deep sleep** entre transmissions: l'ESP32 consumeix ~10 µA en deep sleep.
- **Transmetre poc sovint:** cada 5-15 minuts és un bon equilibri.
- **Usar SF baixos** quan sigui possible: menys temps de transmissió.
- **Llegir sensors només quan calgui:** durant el deep sleep, no caldrà.
- **Apagar perifèrics no usats:** Wi-Fi, Bluetooth, etc.

28.8 Caixa estanca

Per a ús a l'exterior, necessitem una **caixa estanca IP65 o superior**. Opcions:

- **Caixa de plàstic ABS:** barata, fàcil de perforar, IP65 si està ben tancada.
- **Caixa d'alumini:** més cara, més robusta, millor dissipació de calor.
- **Caixa de polièster reforçat amb fibra de vidre:** per a instal·lacions professionals.

A la caixa, cal:

- **Forats per a l'antena:** segellat amb cautxú o silicona.
- **Forats per al sensor d'humitat del sòl:** perquè el sensor sobresurti.
- **Ventilació:** per evitar condensació a l'interior. Una mena de filtre Gore-Tex ajuda.
- **Accés per a la bateria:** per poder-la canviar.

28.9 Posada en marxa

Quan tinguem tot el hardware:

1. **Soldar o connectar** l'antena al mòdul SX1262 (o a la placa).
2. **Carregar el codi** a l'ESP32 (veure capítol 29).
3. **Configurar DevEUI, AppEUI, AppKey** amb els valors generats per TTN.
4. **Verificar** que el node es connecta a la xarxa (veure monitor serie).
5. **Posar-lo a la caixa** estanca.
6. **Muntar-lo al camp** amb bona posició per a l'antena.

28.10 Proves inicials

Abans de posar el node al camp, cal validar:

1. **El node arrenca correctament:** LED encén, missatge al port sèrie.
2. **L'antena està connectada:** sense antena, podem cremar el mòdul!
3. **El node es connecta a la xarxa LoRaWAN:** veure missatges "JOINED" al log.
4. **El node transmet dades:** veure els uplink a la consola de TTN.
5. **Les dades són correctes:** temperatura, humitat, etc., en els rangs esperats.
6. **El deep sleep funciona:** deixar-lo una nit, comprovar que la bateria dura.

28.11 Errors habituals

Error 1: antena no connectada. Síntoma: el mòdul es queda "penjat" o es crema. Solució: SEMPRE connectar l'antena abans d'encendre.

Error 2: tensió d'alimentació incorrecta. Síntoma: el node es reinicia aleatòriament, o no arrenca. Solució: verificar que la font d'alimentació és 3.3V (per al SX1262) i prou potent (almenys 500 mA).

Error 3: pins SPI mal connectats. Síntoma: el node no es comunica amb el SX1262. Solució: verificar les connexions amb un multímetre.

Error 4: deep sleep massa profund. Síntoma: el node no es desperta. Solució: verificar que la interrupció DIO1 està ben connectada.

Error 5: freqüència incorrecta. Síntoma: el node transmet, però el gateway no el sent. Solució: verificar que el node està configurat per a EU868 (no US915 o AS923).

28.12 Resum

En aquest capítol hem vist el hardware del node LoRa: ESP32 + SX1262 + sensors, amb l'esquema elèctric, les opcions d'alimentació, la caixa estanca, i els passos de posada en marxa. Hem après quins sensors són útils per a Hort Osona i com connectar-los. En el proper capítol programarem el node: codi per a ESP32 amb la llibreria LMIC o RadioLib, transmissió de dades en format CayenneLPP, deep sleep, i bones pràctiques de consum.

28.13 Exercicis pràctics

1. Compra una placa ESP32 amb LoRa (Heltec LoRa 32 V3 recomanada).
2. Compra un sensor BME280.
3. Munta el circuit a una protoboard.
4. Connecta l'antena.
5. Carrega un codi d'exemple que llegeixi el BME280 i mostri les dades pel port sèrie.
6. Mesura el consum de corrent amb un multímetre.

7. Implementa deep sleep i mesura el consum en sleep.
8. Documenta al README el hardware del node, els pins, i el consum.

Paraules clau: **ESP32, ESP32-S3, SX1262, Heltec LoRa 32, TTGO LoRa, LILYGO T-Beam, T-Deck, sensor, BME280, SHT31, DHT22, humitat del sòl, LDR, BH1750, pluja, pluviòmetre, balancí, pinout, SPI, I2C, GPIO, ADC, 3.3V, alimentació, LiPo, 18650, TP4056, placa solar, deep sleep, baix consum, caixa estanca, IP65, ABS, alumini, antena, IPEX, U.FL, SMA, soldar, multímetre, consum, mA, µA, deep sleep, watchdog, RTOS, Arduino, ESP-IDF, MicroPython, PlatformIO, IDE, sketch, port sèrie, monitor, debug, ESP-Prog, ESPtool, flashing, firmware, bootloader, partition table, OTA, update, deep sleep wake-up, GPIO wake-up, timer wake-up, sensor reading, I2C scanning, SPI initialization, GPIO configuration, interrupt, ISR, callback, millis(), micros(), delay, Scheduler, FreeRTOS, task, queue, semaphore, mutex, ESP_LOG, ets_printf, log levels, error handling, watchdog timer, brown-out detector, ESP32-C3, ESP32-S2, ESP32-S3, ESP32-C6, variants, deep sleep modes, RTC memory, ULP, coprocessor, ULP wake-up, ext0, ext1, touch wake-up, ulp wake-up, gpio wake-up, timer wake-up, Wi-Fi, Bluetooth, BLE, ESP-NOW, LoRa, SX1276, SX1278, SX1262, SX1268, RFM95, RFM96, RF69, HopeRF, Ai-Thinker, Ra-02, Semtech, modulation, OOK, FSK, GFSK, MSK, GMSK, LoRa, FSK, OOK, packet, radio, antenna, impedance matching, balun, SAW filter, low-pass filter, matching network, RF path, ground plane, EMI, RFI, EMC, compliance, ETSI, FCC, modular approval, regulatory, certification, IP rating, IP65, IP67, IP68, IP69K, UV resistance, temperature range, operating temperature, storage temperature, humidity, condensation, venting, Gore-Tex, vent plug, desiccant, silica gel, anti-condensation heater, condensation prevention, self-heating, solar heating, heat dissipation, thermal management, sun exposure, black box, white box, UV-stabilized, weatherproof, weather-resistant, outdoor enclosure, junction box, ABS enclosure, polycarbonate enclosure, fiberglass enclosure, stainless steel enclosure, aluminum enclosure, mounting bracket, pole mount, wall mount, ground mount, mast mount, weatherhead, cable gland, strain relief, IP68 cable gland, cable entry, sealing, silicone sealant, hot melt glue, epoxy, potting, conformal coating, PCB protection, marine-grade, salt-spray resistant, anti-corrosion, anodized, powder-coated, paint, primer, mounting hardware, U-bolts, hose clamps, screws, anchors, washers, nuts, security screws, tamper-proof, locking, key, lock, hasp, padlock, security, theft prevention, vandalism, lockable enclosure, locking mechanism, latch, clasp, hinge, gasket, O-ring, rubber gasket, foam gasket, IP-rated, IP65, IP66, IP67, IP68, IP69K, NEMA 4, NEMA 4X, NEMA 6, NEMA 6P, outdoor, indoor, indoor/outdoor, dry location, wet location, damp location, hazardous location, Class I Div 1, Class I Div 2, explosion-proof, intrinsically safe, IS, Ex, ATEX, IECEx, hazardous area, gas group, dust group, temperature class, T1, T2, T3, T4, T5, T6, T-class, surface temperature, maximum temperature, ignition temperature, flammable, combustible, hazardous materials, industrial environment, marine environment, coastal environment, salt water, salt spray, corrosion, rust, oxidation, galvanic corrosion, dissimilar metals, electrolytic, electrical isolation, barrier, isolation, insulation, weatherhead, drip loop, cable loop, condensation drain, weep hole, drainage, ventilation, breather, vent plug, Gore-Tex, Goretex, GORE-TEX, expanded PTFE, ePTFE, hydrophobic, oleophobic, breathable, IP-rated vent, screw-in vent, push-in vent, adhesive vent.**

Capitol 29 — Programació del node: ESP32 + LoRaWAN

*“Programar un node LoRa és programació de sistemes encastats amb un extra: el temps d'aire és car i la bateria és finita. Cada byte compta.”**

29.1 Què ha de fer el codi del node

Un node LoRaWAN complet ha de:

1. **Inicialitzar** el hardware (SPI per al SX1262, I2C per als sensors).
2. **Connectar-se a la xarxa LoRaWAN** (procediment de join OTAA).
3. **Llegir els sensors** periòdicament.
4. **Codificar les dades** en un payload binari (CayenneLPP recomanat).
5. **Transmetre** el payload via LoRaWAN.
6. **Entrar en deep sleep** fins a la propera transmissió.
7. **Despertar** i repetir.

Això, en codi, són 100-200 línies d'Arduino o MicroPython. Més del que sembla, menys del que sembla.

29.2 Quina llibreria: LMIC o RadioLib

Hi ha diverses llibreries per a LoRaWAN en ESP32:

MCCI LoRaWAN LMIC (recomanada per a LoRaWAN)

- La versió moderna de la clàssica LMIC d'IBM.
- Molt estable, ben mantinguda.
- Suporta ESP32, ESP8266, STM32, etc.
- API basada en callbacks (pot ser una mica confosa al principi).

RadioLib

- Llibreria moderna que suporta **LoRaWAN, LoRa P2P, FSK, OOK**, etc.
- API més neta que LMIC.
- Suporta moltes plaques i xips (SX126x, SX127x, RFM9x).
- Ideal si volem fer P2P i LoRaWAN amb la mateixa llibreria.

Altres

- **arduino-lmic**: la clàssica, menys mantinguda.

- **TinyLoRa**: minimalista, per a nodes molt petits.
- **Heltec examples**: específics per a les plaques Heltec.

Recomanació: per a LoRaWAN amb ESP32, **MCCI LMIC** o **RadioLib**. Per a P2P, **RadioLib**.

29.3 Configuració de l'entorn Arduino

Si no tens l'entorn Arduino preparat:

1. **Instal·la Arduino IDE** (<https://www.arduino.cc/en/software>) o **PlatformIO** (integrat a VSCode).
2. **Afegeix el suport per a ESP32**: - Arduino IDE: a **File** → **Preferences**, afegeix l'URL https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json a "Additional Board Manager URLs". - Instal·la "esp32" des del Board Manager.
3. **Instal·la les llibreries necessàries** des del Library Manager: - **MCCI LoRaWAN LMIC library** (per a LoRaWAN). - **RadioLib** (alternativa). - **Adafruit BME280 Library** (per al sensor). - **Adafruit Unified Sensor** (dependència).

29.4 Exemple complet: codi per a Heltec LoRa 32 V3 amb BME280

Aquí tens un programa complet que llegeix un BME280 i transmet cada 5 minuts via LoRaWAN:

```
// BernatLab Hort Osona - Node LoRaWAN
// Hardware: Heltec LoRa 32 V3 + BME280
// Llibreria: MCCI LoRaWAN LMIC
// Llicència: CC-BY-SA

#include <Arduino.h>
#include <Wire.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_BME280.h>
#include <lmic.h>
#include <hal/hal.h>
#include <SPI.h>

// Identificadors LoRaWAN (substituir pels teus propis!)
static const u1_t PROGMEM DEVEUI[8] = { 0x70, 0xB3, 0xD5, 0x7E, 0xD0, 0x04, 0xF1, 0xCE };
static const u1_t PROGMEM APPEUI[8] = { 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00 };
static const u1_t PROGMEM APPKEY[16] = {
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00
};

// Pins per a la Heltec LoRa 32 V3
#define PIN_LMIC_NSS 8
#define PIN_LMIC_RST 12
#define PIN_LMIC_DIO0 14
#define PIN_LMIC_DIO1 35
#define PIN_LMIC_DIO2 34

// BME280 (I2C per defecte a la Heltec)
#define SEALEVELPRESSURE_HPA (1013.25)
Adafruit_BME280 bme;

// Configuració de l'objectiu (TTN)
static const u4_t NETID = 0x13; // TTN
```

```

static const u1_t NWKSKEY[16] = { 0 }; // No cal per a OTAA
static const u1_t APPSKEY[16] = { 0 }; // No cal per a OTAA
static const u4_t DEVADDR = 0; // No cal per a OTAA
static const u1_t FREQ_CHNL_HOP = 0; // desactivat (TTN gestiona)
static const u2_t FREQ_HZ_MIN = 868100000;
static const u2_t FREQ_HZ_MAX = 868500000;

const lmic_pinmap lmic_pins = {
    .nss = PIN_LMIC_NSS,
    .rxtx = LMIC_UNUSED_PIN,
    .rst = PIN_LMIC_RST,
    .dio = {PIN_LMIC_DIO0, PIN_LMIC_DIO1, PIN_LMIC_DIO2},
};

// Callbacks de LMIC
void onJoinFailed() {
    Serial.println(F("Join failed"));
}

void onJoinSuccess() {
    Serial.println(F("Join success!"));
    LMIC_setLinkCheckMode(1);
}

void onTransmitted() {
    Serial.println(F("Packet transmitted"));
}

// Tasca periòdica
osjob_t sendjob;

void do_send(osjob_t* j) {
    if (LMIC.opmode & OP_TXRXPEND) {
        Serial.println(F("OP_TXRXPEND, skip"));
    } else {
        // Llegir el sensor
        float temperatura = bme.readTemperature();
        float humitat = bme.readHumidity();
        float pressio = bme.readPressure() / 100.0F; // Pa → hPa

        // Construir el payload CayenneLPP
        uint8_t payload[16];
        int idx = 0;
        // Temperatura (canal 1, valor × 10)
        int16_t t10 = (int16_t)(temperatura * 10);
        payload[idx++] = 0x67; // Tipus: temperatura
        payload[idx++] = 0x01; // Canal: 1
        payload[idx++] = (t10 >> 8) & 0xFF;
        payload[idx++] = t10 & 0xFF;
        // Humitat (canal 2, valor × 2)
        uint16_t h2 = (uint16_t)(humitat * 2);
        payload[idx++] = 0x68; // Tipus: humitat
        payload[idx++] = 0x02; // Canal: 2
        payload[idx++] = (h2 >> 8) & 0xFF;
        payload[idx++] = h2 & 0xFF;
        // Pressió (canal 3, valor en hPa × 10)
        uint16_t p10 = (uint16_t)(pressio * 10);
        payload[idx++] = 0x73; // Tipus: pressió baromètrica
        payload[idx++] = 0x03; // Canal: 3
        payload[idx++] = (p10 >> 8) & 0xFF;
        payload[idx++] = p10 & 0xFF;
        // Voltatge de la bateria (canal 4)
        // ... llegir amb analogRead() i afegir
    }
}

```

```

        // Preparar i enviar
        LMIC_setTxData2(1, payload, idx, 0);
        Serial.print(F("Enviats "));
        Serial.print(idx);
        Serial.println(F(" bytes"));
    }
    // Programar la propera transmissió en 5 minuts
    os_setTimedCallback(&sendjob, os_getTime() + sec2osticks(300), do_send);
}

void setup() {
    Serial.begin(115200);
    delay(2000); // Donar temps a que el port sèrie estigui llest

    // Inicialitzar el BME280
    if (!bme.begin(0x76)) {
        if (!bme.begin(0x77)) {
            Serial.println(F("BME280 no trobat!"));
            while (1) delay(1000);
        }
    }
    Serial.println(F("BME280 OK"));

    // Inicialitzar LMIC
    os_init();
    LMIC_reset();
    LMIC_setClockError(MAX_CLOCK_ERROR * 10 / 100); // Tolerar 10% d'error

    // Configurar DevEUI, AppEUI, AppKey
    LMIC_setSession(0x13, DEVEUI, NWKSKEY, APPSKEY);
    LMIC_startJoin();

    // Programar la primera transmissió
    do_send(nullptr);
}

void loop() {
    os_runloop_once();
}

```

Aquest codi és llarg però estructurat. Repassem les parts importants.

29.5 Estructura del codi

El codi es compon de:

1. **Inclusions:** les llibreries.
2. **Identificadors LoRaWAN:** DevEUI, AppEUI, AppKey.
3. **Configuració de pins:** definits segons la placa.
4. **Inicialització del sensor:** BME280 en el nostre cas.
5. **Inicialització de LMIC:** clau per a LoRaWAN.
6. **Callback de transmissió:** `do_send` que llegeix, codifica, i envia.
7. **Setup:** inicialitza tot i programa la primera transmissió.
8. **Loop:** el bucle principal d'esdeveniments de LMIC.

29.6 CayenneLPP: detalls pràctics

CayenneLPP és el format recomanat per la seva eficiència. Cada dada és:

- 1 byte: **tipus** (0x67 = temperatura, 0x68 = humitat, etc.).
- 1 byte: **canal** (1-255, identifica quin sensor).
- N bytes: **valor** (1-4 bytes segons el tipus).

A la web de CayenneLPP hi ha la llista completa de tipus:

<https://developers.mydevices.com/cayenne/docs/cayenne-lpp/>

Per als nostres sensors:

- **Temperatura (0x67)**: 2 bytes, signed 16-bit, valor $\times 10$. Rang: -3276.7 a +3276.7 °C.
- **Humitat (0x68)**: 2 bytes, unsigned 16-bit, valor $\times 2$. Rang: 0-100 %.
- **Pressió (0x73)**: 2 bytes, unsigned 16-bit, valor $\times 10$. Rang: 0-6553.5 hPa.
- **Llum (0x65)**: 2 bytes, unsigned 16-bit, valor $\times 1$. Rang: 0-65535 lux.
- **Humitat del sòl (0x75)**: 1 byte, unsigned 8-bit, valor $\times 1$. Rang: 0-100 %.
- **Voltatge de bateria (0x74)**: 2 bytes, unsigned 16-bit, valor $\times 100$. Rang: 0-655.35 V.

29.7 El procediment de join OTAA

Quan l'ESP32 arrenca, LMIC fa el procediment de join OTAA automàticament. El procediment és:

1. Genera un **DevNonce** aleatori.
2. Envia un **Join Request** a un canal aleatori de 868 MHz.
3. Escolta el **Join Accept** a RX1 (1 segon després) o RX2 (2 segons).
4. Si rep el Join Accept, deriva les claus de sessió (NwkSKey, AppSKey) i el DevAddr.
5. Si no, espera 30 segons i torna a provar.

El join pot trigar uns segons a completar-se. Mentrestant, el node no pot transmetre dades. Això és normal.

A la consola de TTN, podem veure els join requests a la pàgina del node, secció "Live data".

29.8 Deep sleep: la clau de la bateria

Per a ús amb piles, el deep sleep és fonamental. L'ESP32 pot entrar en deep sleep i consumir només ~10 μ A. Es desperta amb:

- **Temporitzador**: cada X segons/minuts.
- **Interrupció externa**: canvi d'estat d'un pin (per exemple, un polsador).
- **Touchpad**: toc a un pin capacitiu.
- **ULP**: coprocessador que pot fer lectures simples.

Codi per entrar en deep sleep 5 minuts:

```
esp_sleep_enable_timer_wakeup(5 * 60 * 1000000); // 5 minuts en µs
esp_deep_sleep_start();
```

En el nostre cas, podem estructurar el programa així:

```
void loop() {
    // Primer cop: setup() ja ha programat do_send()
    // do_send() transmet i després entra en deep sleep
}

// Modificar do_send() per entrar en deep sleep al final:
void do_send(osjob_t* j) {
    // ... llegir, codificar, transmetre ...
    Serial.println(F("Anant a dormir..."));
    Serial.flush();
    esp_sleep_enable_timer_wakeup(5 * 60 * 1000000);
    esp_deep_sleep_start();
}
```

Així, el cicle és: arrencada → join → llegir → transmetre → dormir 5 min → despertar → llegir → transmetre → dormir → ...

A la pràctica, l'ESP32 triga ~1-2 segons a despertar-se des de deep sleep, fer el join (que potser ja està fet, sinó fer-lo), llegir sensors, transmetre, i tornar a dormir. La majoria del temps, **el node està dormint**.

29.9 Bona gestió de la bateria

Algunes pràctiques:

- **Llegir el voltatge de la bateria** periòdicament i enviar-lo com a dada addicional.
- **Usar el màxim SF possible** quan el node dorm (per a transmissions ràpides).
- **Configurar l'ADR activat** (el NS optimitzarà els paràmetres).
- **Desactivar Wi-Fi i Bluetooth** si no es fan servir (ja ve per defecte en la majoria de llibreries).
- **Usar el mode de baix consum del SX1262** (es desactiva sol quan no transmet).
- **Evitar transmissions amb errors** (reintentar inútilment).

29.10 Logs i debug

Durant el desenvolupament, és important tenir bons logs:

```
#define LMIC_DEBUG_LEVEL 1 // 0 = silenci, 1 = normal, 2 = molt
```

Això ensenya missatges interns de LMIC, útil per veure el procés de join, l'estat de les transmissions, etc.

També podem afegir un **LED** que indiqui l'estat:

```
// LED parpelleja durant el join
// LED fix quan el join és complet
// LED parpelleja durant la transmissió
```

Per a depuració més avançada, podem afegir un **botó** que forci una transmissió immediata.

29.11 Primera càrrega del codi

Per carregar el codi a l'ESP32:

1. **Connectar l'ESP32** al PC per USB.
2. **Seleccionar la placa** a l'Arduino IDE: `Tools` → `Board` → `ESP32 Dev Module` (o `Heltec LoRa 32` si tens aquesta opció).
3. **Seleccionar el port**: `Tools` → `Port` → `COMx` (Windows) o `/dev/ttyUSBx` (Linux).
4. **Pujar el codi**: `Sketch` → `Upload`.
5. **Obrir el monitor sèrie**: `Tools` → `Serial Monitor` a 115200 baud.

Veurem els missatges del node: arrencada, intent de join, transmissió, etc.

29.12 Primera transmissió exitosa

Quan tot funciona, veurem a la consola de TTN:

1. A la pàgina del node, secció "Live data": - "Join request" → "Join accept" - Un uplink amb el payload
1. A la pàgina del gateway: - Un uplink rebut amb RSSI, SNR, etc.
1. A la integració MQTT: - Un missatge JSON al broker.

Si tot això passa, tenim un node LoRaWAN funcional!

29.13 Adaptació a altres sensors

Per afegir un sensor nou (per exemple, un sensor d'humitat del sòl), el codi canvia poc:

```
// Definir el pin
#define SOIL_PIN 34

// A do_send():
int soilValue = analogRead(SOIL_PIN);
float soilPercent = map(soilValue, 0, 4095, 100, 0); // Invertir: 0 = sec, 4095 = mullat
// 0 una corba de calibratge millor:
// float soilPercent = 100.0 * (4095 - soilValue) / 4095.0;

// Codificar CayenneLPP (canal 4)
uint8_t soil = (uint8_t)soilPercent;
payload[idx++] = 0x75; // Tipus: humitat del sòl
payload[idx++] = 0x04; // Canal: 4
payload[idx++] = soil;
```

I afegir-lo al payload formatter de TTN:

```
case 0x75: // Humitat del sòl
    var soil = input.bytes[i++];
    decoded.humitat_sol = soil;
    break;
```


29.14 Versió amb MicroPython

Si prefereixes MicroPython a Arduino, la sintaxi canvia però la lògica és similar. La llibreria LoRaWAN en MicroPython no és tan madura com en C, però per a prototips funciona.

Exemple amb MicroPython i una Heltec LoRa 32 V3:

```
from machine import Pin, I2C, deepsleep
import time
import bme280
import ujson

# Pins
PIN_SS = 8
PIN_RST = 12
PIN_DIO0 = 14

# Inicialitzar BME280
i2c = I2C(0, scl=Pin(22), sda=Pin(21))
bme = bme280.BME280(i2c=i2c)

# Inicialitzar LoRa (amb radio library específica de MicroPython)
# ... (cal instal·lar la llibreria de MicroPython per a SX126x)

while True:
    temperatura, pressio, humitat = bme.read_compensated_data()
    temperatura = temperatura / 100
    humitat = humitat / 1024
    pressio = pressio / 25600

    # Codificar CayenneLPP
    payload = bytes([
        0x67, 0x01, # Temperatura canal 1
        (int(temperatura * 10) >> 8) & 0xFF, int(temperatura * 10) & 0xFF,
        0x68, 0x02, # Humitat canal 2
        (int(humitat * 2) >> 8) & 0xFF, int(humitat * 2) & 0xFF,
        0x73, 0x03, # Pressió canal 3
        (int(pressio * 10) >> 8) & 0xFF, int(pressio * 10) & 0xFF,
    ])

    # Transmetre
    lora.send(payload)

    # Deep sleep 5 minuts
    deepsleep(5 * 60 * 1000)
```

La versió Arduino és més madura i recomanada per a producció. MicroPython és bona per a prototipatge ràpid.

29.15 Errors habituals

Error 1: identificadors incorrectes. Síntoma: el join falla constantment. Solució: verificar DevEUI, AppEUI, AppKey.

Error 2: pins incorrectes. Síntoma: el SX1262 no es comunica. Solució: verificar els pins segons la placa.

Error 3: no configurar la regió. Síntoma: el node transmet però el gateway no el sent. Solució: afegir `LMIC_setRegion(LMIC_region_eu868)` (o similar per a la teva regió).

Error 4: consum excessiu. Síntoma: la bateria dura poc. Solució: entrar en deep sleep correctament, desactivar Wi-Fi i Bluetooth.

Error 5: payload massa gran. Síntoma: transmissió falla o es retarda. Solució: CayenneLPP compacte, no enviar JSON.

29.16 Resum

En aquest capítol hem après a programar un node LoRaWAN amb ESP32 i la llibreria MCCI LMIC. Hem vist el codi complet d'un node que llegeix BME280 i transmet cada 5 minuts en format CayenneLPP. Hem après a configurar el deep sleep per a baix consum, a gestionar la bateria, i a fer el debug amb logs. En el proper capítol veurem com rebre aquestes dades al BernatLab: TTN → Mosquitto → Telegraf → InfluxDB, i com visualitzar-les a Grafana.

29.17 Exercicis pràctics

1. Munta un circuit amb Heltec LoRa 32 V3 + BME280.
2. Instal·la Arduino IDE i les llibreries (LMIC, Adafruit BME280).
3. Configura el codi amb els TEUS DevEUI, AppEUI, AppKey.
4. Carrega el codi a l'ESP32.
5. Mira el monitor sèrie. Hauries de veure "Join success!" en uns segons.
6. A la consola de TTN, comprova que el node apareix com a "Connected".
7. Espera 5 minuts. Hauries de veure un uplink a "Live data".
8. Prova d'afegir un sensor d'humitat del sòl al codi.
9. Mesura el consum de corrent en deep sleep amb un multímetre.
10. Documenta al README el procés de flashing i el consum.

Paraules clau: **programació, codi, Arduino, ESP32, Heltec LoRa 32, LMIC, MCCI, RadioLib, CayenneLPP, payload, deep sleep, esp_sleep_enable, esp_deep_sleep, RTC, RAM persistent, timer wake-up, GPIO wake-up, deep sleep modes, ULP, coprocessador, sensor reading, I2C, SPI, GPIO, ADC, DAC, PWM, timer, interrupt, ISR, callback, FreeRTOS, task, queue, semaphore, mutex, ESP-IDF, Arduino framework, sketch, upload, monitor sèrie, port sèrie, COM, ttyUSB, baud, 115200, log, debug, LMIC_DEBUG_LEVEL, OTAA, ABP, DevEUI, AppEUI, AppKey, NWKSKEY, APPSKEY, DevAddr, DevNonce, join procedure, EU868, regió, data rate, SF, BW, TX power, ack, confirmed, unconfirmed, downlink, RX1, RX2, BME280, temperatura, humitat, pressió, sensor, I2C address, 0x76, 0x77, deep sleep wake-up, timer wake-up, low power, low energy, mAh, bateria, LiPo, 18650, TP4056, charging, battery voltage, ADC, analogRead, voltage divider, MOSFET, fuel gauge, INA219, INA260, coulomb counter, battery health, charge cycle, deep discharge, over-discharge, undervoltage, low battery, power management, sleep current, active current, average current, deep sleep current, wake-up time, boot time, startup time, frequency, clock, ESP32 clock, crystal, oscillator, RTC, real-time clock, external RTC, DS3231, PCF8523, time sync, NTP, time server, time protocol, timestamp, log timestamp, debug log, Serial.println, sprintf, format string, buffer, overflow, memory, heap, stack, fragmentation, watchdog, brown-out, panic, error handling, exception, fault, GDB,**

JTAG, debug, OCD, ESP-Prog, ESPTool, esptool.py, flash tool, firmware, bootloader, partition table, OTA, update over the air, FOTA, firmware update, secure boot, flash encryption, NVS, preferences, ESP32 preferences, LittleFS, SPIFFS, file system, SD card, microSD, data logging, local storage, send to server, queue, batch, retransmission, reliable, MQTT-SN, CoAP, LwM2M, lightweight, IoT protocol, application protocol, network protocol, transport protocol, MAC protocol, modulation, spreading, coding, error correction, FEC, interleaving, whitening, channel coding, source coding, compression, encryption, AES, CCM, integrity, MAC, message authentication, replay attack, sequence number, frame counter, MIC, message integrity code, frame integrity, data integrity, OTAA, ABP, session key, network key, application key, root key, join key, NwkKey, AppKey, MCKE, multicast key, multicast, group, multicast setup, FUOTA, firmware update over the air, multicast firmware update, class B, class C, downlink, RX1, RX2, receive window, RX1 delay, RX2 delay, dwell time, maximum payload, dwell time limitation, EU868 dwell time, ETSI EN 300 220, regional parameters, LoRaWAN regional parameters, RP001, RP002, LoRaWAN specification, LoRaWAN 1.0, LoRaWAN 1.0.2, LoRaWAN 1.0.3, LoRaWAN 1.0.4, LoRaWAN 1.1, MAC specification, regional parameters document, Lora Alliance, TS001, TS002, TS003, TS004, TS005, TS006, TS007, TS008, TS009, TS010, TS011, LoRaWAN L2 specification, layer 2, MAC layer, physical layer, PHY layer, application layer, application server, network server, join server, application session, network session, session context, MAC commands, fopts, FOpts, FOptsLen, FCtrl, frame control, frame header, FPort, FRMPayload, payload, MIC, message integrity check, B0, B1, B2, encryption, decryption, AES-128, AES-CMAC, CMAC, MIC calculation, MIC verification, packet validation, packet filtering, deduplication, desduplicació, replay protection, replay attack, frame counter, FCnt, FCntUp, FCntDown, frm_payload_size, payload size, maximum payload size, dwell time, time on air, airtime, transmission time, duty cycle, EU868 duty cycle, 1% duty cycle, 0.1% duty cycle, 10% duty cycle, sub-banda, 868.0-868.6, 868.7-869.2, 869.4-869.65, ETSI, ERC Recommendation 70-03, EN 300 220, ETSI EN 300 220-1, ETSI EN 300 220-2, regulator, telecom regulator, bandwidth, 125 kHz, 250 kHz, 500 kHz, channel, 868.1, 868.3, 868.5, 867.1, 867.3, 867.5, 867.7, 867.9, frequency plan, EU868 frequency plan, RX2 channel, RX2 data rate, default RX2, network server RX2, second receive window, RX2 frequency, RX2 DR, RX2 SF, RX2 BW, RX1 channel, RX1 DR, RX1 SF, RX1 BW, RX1 data rate, RX1 delay, receive delay, transmission timing, hop limit, MAC command, ADR, ADR bit, ADRACKReq, ADRACKCnt, LinkADRReq, LinkADRAns, DutyCycleReq, DutyCycleAns, RXParamSetupReq, RXParamSetupAns, DevStatusReq, DevStatusAns, NewChannelReq, NewChannelAns, RXTimingSetupReq, RXTimingSetupAns, Class B, ping slot, beacon, BCN, beacon frequency, beacon period, ping slot period, multicast, MCKS, multicast address, group address, FUOTA, fragmentation, file fragmentation, fragIndex, fragCount, MIC, integrity check, signed fragment, signature, EUI-64, EUI-48, dev nonce, join nonce, AppNonce, DevNonce, JoinNonce, LoRaWAN 1.0, LoRaWAN 1.1, LoRaWAN 1.0.4, LoRaWAN 1.0.3, LoRaWAN 1.0.2, application root key, root key, application key, network key, NwkKey, SNwkSIntKey, FNwkSIntKey, NwkSEncKey, AppSKey, root key, derivation, session key derivation, key derivation, KDF, key derivation function, session context, session state, security context, join server, network server, application server, separated AS, separated NS, separated JS, server architecture, distributed, monolithic, integrated, key management, key rotation, root key rotation, secure element, secure storage, hardware secure element, ATECC608, ATECC608A, secure element, ECC, elliptic curve, ECDSA, ECDH, signature, verification, secure boot, secure firmware, hardware root of trust, immutable bootloader, signed firmware, firmware signature, secure firmware update, FOTA, FUOTA, secure FOTA.

Capítol 30 — Recepció al BernatLab: de TTN a InfluxDB

"El node ha transmès, el gateway ha escoltat, TTN ha desxifrat. Ara falta la part que ens interessa: que les dades aterrin al Grafana i a la web."

30.1 El pipeline complet

Quan un node LoRaWAN transmet, les dades fan el següent camí:



Aquest pipeline ja el coneixem del Mòdul 2, però ara veurem els detalls específics per a dades LoRa.

30.2 El que rep Mosquitto

Quan TTN ha desxifrat el payload d'un node, el publica al broker MQTT. El missatge té aquesta estructura JSON:

```

{
  "end_device_ids": {
    "device_id": "eui-70b3d57ed004f1ce",
    "application_ids": {
      "application_id": "hort-osona-bernat"
    }
  },
  "dev_eui": "70B3D57ED004F1CE",
  "join_eui": "0000000000000000",
  "received_at": "2026-07-08T12:34:56.789Z",
  "uplink_message": {
    "f_port": 2,
    "f_cnt": 42,
    "frm_payload": "GAEABbYBmAu4",
    "decoded_payload": {
      "temperatura": 25.5,
      "humitat": 91,
      "pressio": 1013
    }
  },
  "rx_metadata": [{
    "gateway_ids": {
      "gateway_id": "raspi-hortosona-gw"
    }
  },
  "rssi": -67,
  "snr": 9.5
}

```

```

    }},
    "settings": {
      "data_rate": {
        "lorawan": {
          "bandwidth": 125000,
          "spreading_factor": 9
        }
      },
      "frequency": "868100000"
    }
  }
}

```

Com veiem, **el payload decodificat ja ve en JSON** (gràcies al payload formatter que hem configurat a TTN). També tenim metadades útils: RSSI, SNR, data rate, etc.

30.3 Subscriure's manualment per veure-ho

Per veure els missatges en directe, ens podem subscriure al broker:

```
mosquitto_sub -h 100.115.134.76 -t "v3/#" -v -u bernat -P CONTRASENYA
```

Això ensenya tots els missatges que arriben. Si veiem alguna cosa, vol dir que TTN s'està connectant correctament al nostre broker.

30.4 Configurar Telegraf per rebre dades LoRa

Ara ve la part interessant: configurar Telegraf perquè consumeixi els missatges de TTN i els escrigui a InfluxDB.

A `/home/bernat/homelab/compose/iot/telegraf.conf`:

```

[agent]
  interval = "30s"
  flush_interval = "30s"

# Input: escoltem MQTT per missatges de TTN
[[inputs.mqtt_consumer]]
  servers = ["tcp://mosquitto:1883"]
  username = "telegraf"
  password = "${MQTT_PASSWORD}"
  client_id = "telegraf-ttn-bridge"
  topics = ["v3/+/@/devices/+/up"]
  data_format = "json"
  json_time_key = "received_at"
  json_time_format = "2006-01-02T15:04:05Z"
  tag_keys = ["device_id", "application_id", "dev_eui"]

# Tag a partir d'una ruta JSON: el data rate i el SF
[[inputs.mqtt_consumer.json_fields]]
  path = "uplink_message.settings.data_rate.lorawan.spreading_factor"
  name = "spreading_factor"
  type = "integer"
[[inputs.mqtt_consumer.json_fields]]
  path = "uplink_message.settings.data_rate.lorawan.bandwidth"
  name = "bandwidth"
  type = "integer"
[[inputs.mqtt_consumer.json_fields]]
  path = "uplink_message.rx_metadata[0].rssi"
  name = "rssi"
  type = "integer"

```

```

[[inputs.mqtt_consumer.json_fields]]
  path = "uplink_message.rx_metadata[0].snr"
  name = "snr"
  type = "float"
[[inputs.mqtt_consumer.json_fields]]
  path = "uplink_message.f_cnt"
  name = "fcnt"
  type = "integer"

# Camps de la payload (cayenne LPP ja desxifrat per TTN)
[[inputs.mqtt_consumer.json_fields]]
  path = "uplink_message.decoded_payload.temperatura"
  name = "temperatura"
  type = "float"
[[inputs.mqtt_consumer.json_fields]]
  path = "uplink_message.decoded_payload.humitat"
  name = "humitat"
  type = "float"
[[inputs.mqtt_consumer.json_fields]]
  path = "uplink_message.decoded_payload.pressio"
  name = "pressio"
  type = "float"
[[inputs.mqtt_consumer.json_fields]]
  path = "uplink_message.decoded_payload.humitat_sol"
  name = "humitat_sol"
  type = "float"
[[inputs.mqtt_consumer.json_fields]]
  path = "uplink_message.decoded_payload.lux"
  name = "lux"
  type = "integer"
[[inputs.mqtt_consumer.json_fields]]
  path = "uplink_message.decoded_payload.voltatge_bateria"
  name = "voltatge_bateria"
  type = "float"

# Output: InfluxDB v2
[[outputs.influxdb_v2]]
  urls = ["http://influxdb:8086"]
  token = "${INFLUX_TOKEN}"
  org = "bernatlab"
  bucket = "hort-osona"
  timeout = "10s"
  content_encoding = "gzip"
# Mesura comuna per a tots els punts
measurement = "sensor_lora"

```

Aquesta configuració:

- Escolta tots els missatges de TTN ([v3/+/@/devices/+/up](#)).
- Extreu els tags: `device_id`, `application_id`, `dev_eui`.
- Extreu les metadades: SF, BW, RSSI, SNR, frame counter.
- Extreu les dades: temperatura, humitat, pressió, etc.
- Usa el temps de `received_at` de TTN com a timestamp.
- Escriu a InfluxDB al bucket `hort-osona`, amb mesura `sensor_lora`.

30.5 Verificar que Telegraf rep dades

Podem mirar els logs:

```
docker compose logs -f telegraf
```

Si tot funciona, veurem línies com:

```
2026-07-08T12:34:56Z I! Loaded inputs: mqtt_consumer
2026-07-08T12:34:56Z I! Loaded outputs: influxdb_v2
2026-07-08T12:34:56Z I! [agent] Start: agent started
```

I quan arribi una transmissió, veurem l'escriptura a InfluxDB.

També podem consultar directament a InfluxDB:

```
influx query '
from(bucket: "hort-osona")
  |> range(start: -1h)
  |> filter(fn: (r) => r._measurement == "sensor_lora")
  |> filter(fn: (r) => r._field == "temperatura")
' --org bernatlab --token TOKEN
```

Si veiem dades, tot funciona.

30.6 Configurar Grafana

A Grafana (que ja tenim configurat al M2), creem o importem un dashboard específic per a sensors LoRa.

Un dashboard bàsic tindria:

1. **Panell "Temperatura per zona"**: gràfica de línies, últimes 24 h.
2. **Panell "Humitat per zona"**: similar.
3. **Panell "Humitat del sòl"**: gràfica específica.
4. **Panell "RSSI / SNR"**: gràfiques de qualitat del senyal.
5. **Panell "Bateria"**: gràfica del voltatge de la bateria.
6. **Panell "Estat del node"**: taula amb l'últim valor, RSSI, SNR, etc.
7. **Alerta visual**: si la temperatura baixa de 2 °C, el panell es posa vermell.

Crear el dashboard és similar al que vam fer al Mòdul 2 (Capítol 19). Aquí només afegim les dades específiques de LoRa.

30.7 Alertes amb Grafana

A més de les alertes visuals, podem configurar alertes reals:

- **Temperatura massa baixa** (gelada imminent): alerta per Telegram.
- **Bateria baixa**: alerta per Telegram.
- **Node inactiu**: si no hem rebut cap uplink en 30 minuts, alerta.
- **RSSI massa baix**: indica que alguna cosa ha canviat al camp.

Aquestes alertes les creem a Grafana → Alerting → Alert rules.

30.8 Configurar Node-RED per processar

A Node-RED, podem afegir fluxos específics per a LoRa:

- **Flux 1: filtre de qualitat.** Si RSSI < -110 dBm, alerta.
- **Flux 2: detecció de gelada.** Si temperatura < 2 °C, alerta per Telegram.
- **Flux 3: reg automàtic.** Si humitat del sòl < 30 % durant 30 min, enviar comanda a la vàlvula de reg.
- **Flux 4: resum diari.** Cada matí, enviar un resum a Telegram amb les últimes lectures.

Ja tenim Node-RED instal·lat (M2, Capítol 17), només cal afegir els fluxos nous.

30.9 Exemple de flux Node-RED per alerta de gelada

Un flux senzill a Node-RED que escolta els missatges de TTN i alerta si la temperatura baixa de 2 °C:

1. **Node mqtt in**: subscriu a v3/+/@/devices/+/up.
2. **Node function**: filtra els missatges amb temperatura < 2 °C.
3. **Node telegram sender**: envia l'alerta.

Codi del node function:

```
// Filtrar missatges amb temperatura baixa
const payload = msg.payload;

if (!payload.uplink_message || !payload.uplink_message.decoded_payload) {
  return null;
}

const temp = payload.uplink_message.decoded_payload.temperatura;
if (typeof temp !== 'number' || temp >= 2) {
  return null;
}

const deviceId = payload.end_device_ids.device_id;
const rssi = payload.uplink_message.rx_metadata[0]?.rssi || 'N/A';

msg.payload = `*■ ALERTA GELADA\n\nNode: ${deviceId}\nTemperatura: ${temp}°C\nRSSI: ${rssi} dBm\nHora: $
return msg;
```

Aquest flux ja s'encarrega de la detecció i l'alerta.

30.10 Publicar a la web Hort Osona

Per integrar les dades LoRa a la web pública, l'API FastAPI (M2, Capítol 20) ja està preparada.

Només cal afegir endpoints específics per a sensors LoRa:

```
@router.get("/lora/zones", response_model=list[str])
def llistar_zones_lora():
    """Llista les zones (devices) amb sensors LoRa actius."""
    query = ''
    import "influxdata/influxdb/schema"
    schema.tagValues(bucket: "hort-osona", tag: "device_id")
    ''
    result = query_api.query(query)
    return [table.values[0] for table in result]
```



```
@router.get("/lora/{device_id}/latest")
def ultimes_dades_lora(device_id: str):
    """Retorna les últimes dades d'un sensor LoRa."""
    query = f'''
    from(bucket: "hort-osona")
      |> range(start: -1h)
      |> filter(fn: (r) => r._measurement == "sensor_lora")
      |> filter(fn: (r) => r.device_id == "{device_id}")
      |> last()
    '''
    # ... processar el resultat i retornar JSON
```

La web Hort Osona pot consumir aquests endpoints per mostrar dades en temps quasi-real.

30.11 Còpies de seguretat de les dades LoRa

Les dades dels sensors LoRa són valuoses (representen mesos d'observació). Per tant, cal fer-ne còpia de seguretat:

- **InfluxDB:** la còpia regular d'InfluxDB (M2, Capítol 22) ja inclou aquestes dades.
- **TTN:** TTN ja en té còpia al núvol (gratuït).
- **Local:** podem exportar les dades periòdicament amb una tasca programada.

30.12 Monitoratge del pipeline complet

Per saber que tot funciona, monitorarem:

1. **Concentrator:** monitor HTTP al port 3001 (M3, Capítol 27).
2. **Mosquitto:** monitor TCP al port 1883 (M2, Capítol 13).
3. **Telegraf:** monitor HTTP al port 8086 (no té interfície web, però podem fer un `docker ps`).
4. **InfluxDB:** monitor HTTP al port 8086 (amb `/health`).
5. **Grafana:** monitor HTTP al port 3000.

A Uptime Kuma, afegim tots aquests monitors. Si algun falla, alerta per Telegram.

30.13 Proves d'integració

Quan tot estigui en marxa, validarem el pipeline amb una prova end-to-end:

1. **Node transmet:** veure al log del node "Packet transmitted".
2. **Gateway rep:** veure a la consola de TTN l'uplink.
3. **TTN publica a MQTT:** veure el missatge amb `mosquitto_sub`.
4. **Telegraf escriu a InfluxDB:** veure la dada amb `influx query`.
5. **Grafana mostra:** veure la gràfica actualitzada.
6. **Web pública:** veure la dada al frontend d'Hort Osona.
7. **Node-RED alerta:** si la temperatura és baixa, rebre un missatge a Telegram.

Si tot això funciona, tenim el pipeline complet.

30.14 Resum

En aquest capítol hem après a connectar el pipeline LoRa amb la resta del BernatLab. Hem vist com configurar Telegraf per rebre els missatges de TTN via MQTT, com guardar les dades a InfluxDB, com visualitzar-les a Grafana, com processar-les amb Node-RED, i com exposar-les a través de l'API per a la web pública. Hem après a monitorar tot el pipeline amb Uptime Kuma. En el proper capítol veurem el cas alternatiu: LoRa P2P amb SX1262, sense LoRaWAN ni TTN, útil per a un sol node o per a xarxes privades.

30.15 Exercicis pràctics

1. Desplega la nova configuració de Telegraf per rebre dades de TTN.
2. Configura el payload formatter CayenneLPP a la consola de TTN.
3. Configura la integració MQTT a TTN apuntant al broker del BernatLab.
4. Verifica que les dades arriben a InfluxDB amb `influx query`.
5. Crea un panell a Grafana per visualitzar la temperatura d'un node.
6. Crea una alerta de Grafana per temperatura baixa.
7. Afegeix un flux a Node-RED que processi les dades LoRa.
8. Afegeix un monitor a Uptime Kuma per al gateway LoRa.
9. Documenta al README el pipeline complet amb un esquema.

Paraules clau: **TTN, The Things Stack, MQTT, Mosquitto, Telegraf, InfluxDB, Grafana, Node-RED, FastAPI, API, web pública, CayenneLPP, payload formatter, decoder, telemetry, uplink, downlink, RSSI, SNR, frame counter, FCnt, FPort, data rate, spreading factor, bandwidth, time on air, EU868, frequency plan, channels, duty cycle, ADR, device EUI, application EUI, join, OTAA, ABP, session, dev nonce, join nonce, XSS, integració, broker, Mosquitto, publicador, subscriber, wildcard, v3/, application_id, device_id, end device, application, gateway, concentrator, packet forwarder, server address, EU1, NAM1, AU1, AS923, US915, region, RSSI, SNR, signal quality, monitoring, alerting, Uptime Kuma, Telegram, alert rules, dashboard, panel, gauge, time series, bar chart, table, threshold, alert visual, alert real, contact point, alert evaluation, alert manager, Prometheus, alertmanager, InfluxDB, line protocol, bucket, org, retention, schema, measurements, tags, fields, time series database, TSDB, continuous query, task, aggregate, mean, max, min, last, range filter, tag filter, field filter, group by, window, windowPeriod, timeRangeStart, timeRangeStop, query, InfluxQL, Flux, query builder, Data Explorer, task scheduler, cron, InfluxDB API, InfluxDB CLI, influx command, influx query, influx write, influx bucket, influx auth, influx task, backup, restore, retention policy, downsampling, performance, index, TSI, time series index, cardinality, series cardinality, tag cardinality, series limit, query performance, Flux performance, query optimization, predicate pushdown, predicate, pushdown, caching, query cache, partition, shard, shard duration, shard group, retention enforcement, drop, series, expired series, series expiry, query history, query log, debug, info, error, warning, structured logging, JSON logging, log destination, stderr, stdout, file, syslog, journald, network, log shipping, Loki, Grafana Loki, log aggregation, log query, logQL, alerts, recording rules, alert rules, hot, warm, cold, tier, storage, S3, GCS, Azure, local, disk, memory, RAM, heap, OOM,**

out of memory, garbage collection, GC, performance, latency, throughput, write throughput, read throughput, queries per second, points per second, series per second, ingestion, ingestion rate, cardinality budget, cardinality, tag values, tag keys, schema, schema exploration, schema management, schema design, tag design, field design, data model, time series data model, time series best practices, naming conventions, tag naming, field naming, measurement naming, bucket naming, retention, downsampling, aggregation, continuous query, Flux task, scheduled task, cron task, InfluxDB tasks, task runs, task history, task options, task configuration, task management, task monitoring, task alerts.

Capítol 31 — LoRa P2P amb SX1262: xarxes privades sense TTN

*“Hi ha una altra manera de fer anar LoRa: sense servidor, sense TTN, sense ningú al mig. Dos mòduls parlen i punt. Més simple, menys potent, i de vegades exactament el que necessites.”**

31.1 Quan té sentit LoRa P2P

Al capítol 25 vàrem veure l'arbre de decisió. LoRa P2P (punt a punt) té sentit quan:

- Tens un sol node i un sol receptor.
- Vols privacitat total (cap dada no passa per tercers).
- Estàs fent experiments amb la capa física.
- El volum de dades és tan petit que TTN no val la pena.

Per a Hort Osona no és la millor opció, però entenem que algunes situacions particulars la fan útil. Aquest capítol cobreix com fer-ho, amb exemples reals.

31.2 Què és LoRa P2P

En mode P2P, dos mòduls SX1262 es comuniquen directament. Sense gateway, sense network server. Tu defineixes:

- La freqüència (868.1 MHz, 868.3 MHz, etc.).
- El spreading factor (SF7-SF12).
- La amplada de banda (125, 250, 500 kHz).
- La potència de transmissió.
- El format del payload.
- Si hi ha ACK o no.
- Si hi ha xifrat o no.

Tot és a les teves mans. La llibreria **RadioLib** de jgromes és la millor eina per a P2P: moderna, ben documentada, suporta SX126x, SX127x, RFM9x.

31.3 Diferències amb LoRaWAN

Característica	LoRaWAN	LoRa P2P
Network server	Necessari (TTN, ChirpStack)	No cal
Identificadors	DevEUI, AppEUI, AppKey	Cap (o propis)
Autenticació	Xifrada amb AppKey	Opcional, manual
Adreçament	DevAddr, multi-node	Cap (només 1 a 1)
Downlink	Estàndard	Manual, complicat
ADR	Automàtic	Manual
Consum de memòria	~30-50 KB de codi	~5-10 KB
Complexitat	Mitjana-alta	Baixa

P2P és més simple, però tot el que abans feia el network server, ara ho has de fer tu.

31.4 Hardware: el que necessitem

Per a P2P, el hardware és exactament el mateix que per a LoRaWAN:

- Dos mòduls SX1262 (un com a node, un com a receptor).
- Un microcontrolador per a cada mòdul (ESP32, per exemple).
- Antenes 868 MHz.
- Alimentació.

La diferència és al software: amb P2P no cal TTN ni gateway.

31.5 Receptor: SX1262 connectat a la Raspberry

El receptor és un mòdul SX1262 connectat directament a la Raspberry per SPI. Hi ha diverses opcions:

Opció 1: breakout board SX1262 + Raspberry

Connectem un breakout SX1262 als pins SPI de la Raspberry. Calen cables (no és tan net com un gateway SX1302).

Opció 2: LoRa HAT per a Raspberry

Hi ha HATs per a Raspberry amb SX1262, com el Waveshare SX1262 LoRa HAT. Són barats (~15-25 €) i s'instal·len com un gateway però amb menys capacitats.

Opció 3: USB LoRa dongle

Hi ha dongles USB amb SX1262 que es connecten a un port USB. Es comuniquen per serial. Molt còmode per a prototipatge.

Al BernatLab, recomanem l'opció 2 (HAT) per simplicitat. Si volem més versatilitat, l'opció 3 (USB dongle) és la més còmoda.

31.6 Codi del receptor en Python

A la Raspberry, un script Python que escolta els missatges P2P:

```
#!/usr/bin/env python3
"""
receptor_lora_p2p.py
Rep missatges d'un node LoRa P2P amb SX1262 i els publica a MQTT.

Hardware: Waveshare SX1262 LoRa HAT a la Raspberry Pi 4.
"""

import json
import time
import struct
import paho.mqtt.client as mqtt
from datetime import datetime
from SX126x import SX126x

# Configuració LoRa
LORA_FREQ = 868100000 # 868.1 MHz
LORA_SF = 9 # Spreading Factor
LORA_BW = 125000 # 125 kHz
LORA_CR = 5 # Coding Rate 4/5
LORA_TX_POWER = 14 # dBm

# MQTT
MQTT_BROKER = "localhost"
MQTT_PORT = 1883
MQTT_USER = "lora-receptor"
MQTT_PASS = "password"
MQTT_TOPIC = "lora/hort/#"

# Inicialitzar el SX1262
lora = SX126x(
    freq_hz=LORA_FREQ,
    spreading_factor=LORA_SF,
    bandwidth=LORA_BW,
    coding_rate=LORA_CR,
    tx_power=LORA_TX_POWER,
)

# Inicialitzar MQTT
client = mqtt.Client()
client.username_pw_set(MQTT_USER, MQTT_PASS)
client.connect(MQTT_BROKER, MQTT_PORT, 60)
client.loop_start()

def decode_payload(payload: bytes) -> dict:
    """
    Decodifica un payload CayenneLPP.

    Format esperat:
    - 0x67 0x01 [2 bytes] : Temperatura (canal 1, valor × 10, signed)
    - 0x68 0x02 [2 bytes] : Humitat (canal 2, valor × 2, unsigned)
    - 0x73 0x03 [2 bytes] : Pressió (canal 3, valor × 10, unsigned)
    """
    decoded = {}
```

```

i = 0
while i < len(payload):
    if i + 1 >= len(payload):
        break
    tipo = payload[i]
    canal = payload[i + 1]
    i += 2
    if tipo == 0x67: # Temperatura
        if i + 1 >= len(payload):
            break
        valor = struct.unpack(">h", payload[i:i + 2])[0]
        decoded["temperatura"] = valor / 10.0
        i += 2
    elif tipo == 0x68: # Humitat
        if i + 1 >= len(payload):
            break
        valor = struct.unpack(">H", payload[i:i + 2])[0]
        decoded["humitat"] = valor / 2.0
        i += 2
    elif tipo == 0x73: # Pressió
        if i + 1 >= len(payload):
            break
        valor = struct.unpack(">H", payload[i:i + 2])[0]
        decoded["pressio"] = valor / 10.0
        i += 2
    elif tipo == 0x75: # Humitat del sòl
        if i >= len(payload):
            break
        decoded["humitat_sol"] = payload[i]
        i += 1
    else:
        # Tipus desconegut, aturar
        break
return decoded

def publicar_a_mqtt(decoded: dict, rssi: int, snr: float):
    """Publica les dades decodificades al broker MQTT."""
    payload = {
        "timestamp": datetime.utcnow().isoformat() + "Z",
        "rssi": rssi,
        "snr": snr,
        **decoded,
    }
    msg = json.dumps(payload)
    # Topic basat en el tipus de dada
    for key in decoded:
        topic = f"lora/hort/{key}"
        client.publish(topic, msg, retain=True)
    # També publiquem el payload complet
    client.publish("lora/hort/all", msg, retain=True)
    print(f"Publicat: {msg}")

def main():
    print(f"Receptor LoRa P2P a {LORA_FREQ/1e6} MHz, SF{LORA_SF}, BW {LORA_BW/1e3} kHz")
    print("Esperant transmissions...")

    while True:
        try:
            data, rssi, snr = lora.receive(timeout_ms=10000)
            if data:
                print(f"Rebut ({len(data)} bytes), RSSI={rssi}, SNR={snr}")
                decoded = decode_payload(data)

```

```

        if decoded:
            publicar_a_mqtt(decoded, rssi, snr)
        else:
            print(f"Payload no reconegut: {data.hex()}")
    except KeyboardInterrupt:
        print("\nAturant...")
        break
    except Exception as e:
        print(f"Error: {e}")
        time.sleep(1)

if __name__ == "__main__":
    main()

```

Aquest codi:

- Inicialitza el SX1262 amb els paràmetres adequats.
- Escolta transmissions en bucle.
- Decodifica payloads CayenneLPP.
- Publica a MQTT per integrar-se amb la resta del BernatLab.

31.7 Codi del node en Arduino (ESP32 + SX1262)

El node és similar al que vam veure al capítol 29, però usant RadioLib i mode P2P:

```

// BernatLab Hort Osona - Node LoRa P2P
// Hardware: Heltec LoRa 32 V3 + BME280
// Llibreria: RadioLib

#include <Arduino.h>
#include <Wire.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_BME280.h>
#include <RadioLib.h>

// Pins per a la Heltec LoRa 32 V3
#define LORA_NSS 8
#define LORA_RST 12
#define LORA_DI01 14
#define LORA_BUSY 13

// SX1262 amb RadioLib
SX1262 lora = new Module(LORA_NSS, LORA_DI01, LORA_RST, LORA_BUSY);

// BME280
Adafruit_BME280 bme;

// Paràmetres P2P
#define LORA_FREQ 868.1
#define LORA_BW 125.0
#define LORA_SF 9
#define LORA_CR 5
#define LORA_TX_POWER 14

// Banderes
bool transmissio_OK = false;
volatile bool transmissio_completada = false;

void transmissio_completada_callback() {

```



```

    transmissio_completada = true;
}

void setup() {
    Serial.begin(115200);
    delay(2000);

    // Inicialitzar BME280
    if (!bme.begin(0x76) && !bme.begin(0x77)) {
        Serial.println(F("BME280 no trobat"));
        while (1) delay(1000);
    }

    // Inicialitzar SX1262 amb RadioLib
    Serial.print(F("Inicialitzant SX1262... "));
    int state = lora.begin();
    if (state != RADIOLIB_ERR_NONE) {
        Serial.print(F("Error: "));
        Serial.println(state);
        while (1) delay(1000);
    }
    Serial.println(F("OK"));

    // Configurar paràmetres
    lora.setFrequency(LORA_FREQ);
    lora.setSpreadingFactor(LORA_SF);
    lora.setBandwidth(LORA_BW);
    lora.setCodingRate(LORA_CR);
    lora.setOutputPower(LORA_TX_POWER);

    // Callback
    lora.setPacketSentAction(transmissio_completada_callback);
}

void loop() {
    // Llegir sensor
    float temperatura = bme.readTemperature();
    float humitat = bme.readHumidity();
    float pressio = bme.readPressure() / 100.0;

    // Codificar CayenneLPP
    uint8_t payload[16];
    int idx = 0;
    int16_t t10 = (int16_t)(temperatura * 10);
    payload[idx++] = 0x67;
    payload[idx++] = 0x01;
    payload[idx++] = (t10 >> 8) & 0xFF;
    payload[idx++] = t10 & 0xFF;
    uint16_t h2 = (uint16_t)(humitat * 2);
    payload[idx++] = 0x68;
    payload[idx++] = 0x02;
    payload[idx++] = (h2 >> 8) & 0xFF;
    payload[idx++] = h2 & 0xFF;
    uint16_t p10 = (uint16_t)(pressio * 10);
    payload[idx++] = 0x73;
    payload[idx++] = 0x03;
    payload[idx++] = (p10 >> 8) & 0xFF;
    payload[idx++] = p10 & 0xFF;

    // Transmetre
    Serial.print(F("Enviant... "));
    transmissio_completada = false;
    int state = lora.startTransmit(payload, idx);
    if (state != RADIOLIB_ERR_NONE) {

```

```

        Serial.print(F("Error: "));
        Serial.println(state);
    } else {
        // Esperar la transmissió
        unsigned long start = millis();
        while (!transmisio_completada && millis() - start < 5000) {
            lora.yield();
        }
        if (transmisio_completada) {
            Serial.println(F("OK"));
        } else {
            Serial.println(F("Timeout"));
        }
    }
}

// Deep sleep 5 minuts
Serial.println(F("Dormint..."));
Serial.flush();
esp_sleep_enable_timer_wakeup(5 * 60 * 1000000);
esp_deep_sleep_start();
}

```

31.8 Avantatges i inconvenients del P2P

Avantatges

- **Simplicitat.** No cal configurar TTN, no cal entendre DevEUI, AppEUI, etc.
- **Privacitat total.** Cap dada no surt del teu entorn.
- **Cost més baix.** No cal gateway SX1302 (més car), un SX1262 a la Raspberry n'hi ha prou.
- **Flexibilitat total.** Tu tries el format del payload, les capçaleres, el xifrat.

Inconvenients

- **Escalabilitat limitada.** Dos mòduls, no més. Per a més, cal reimplementar la xarxa.
- **Sense roaming.** Si el node es mou, cal reconfigurar.
- **Sense ADR.** Cal optimitzar manualment.
- **Sense eines de monitoratge.** TTN ensenya gràfiques, RSSI, etc. En P2P, cal implementar-ho.
- **Cal gestionar errors.** En LoRaWAN, el NS desduplica i retransmet. En P2P, cal fer-ho manualment.

31.9 Com afegir reconeixement (ACK) en P2P

LoRaWAN té el seu mecanisme d'ACK. En P2P, podem implementar-lo manualment:

```

// Al node: després de transmetre, escoltar un ACK durant 2 segons
lora.startTransmit(payload, idx);
// ... esperar la transmissió ...
// Escoltar un ACK durant RX1 (1s) i RX2 (2s)
String ack;
int state = lora.receive(ack, 0, 0, 2000);
if (state == RADIOLIB_ERR_NONE) {
    Serial.println("ACK rebut");
} else {

```

```
    Serial.println("Sense ACK, retransmetre");
}
```

Al receptor:

```
# Quan rebem un missatge, enviem un ACK
def enviar_ack(received_payload):
    ack_payload = bytes([0xFF, 0x00, 0x01]) # Tipus ACK, longitud, OK
    lora.transmit(ack_payload)
```

Això afegeix complexitat, però permet saber si el missatge ha arribat.

31.10 Com xifrar en P2P

LoRaWAN xifra amb AES-128. En P2P, podem fer el mateix amb qualsevol llibreria AES:

```
#include <mbedtls/aes.h>

uint8_t key[16] = { /* 16 bytes de clau */ };
uint8_t iv[16] = { /* IV aleatori */ };
uint8_t plaintext[16];
uint8_t ciphertext[16];

mbedtls_aes_context aes;
mbedtls_aes_setkey_enc(&aes, key, 128);
mbedtls_aes_crypt_cbc(&aes, MBEDTLS_AES_ENCRYPT, 16, iv, plaintext, ciphertext);
```

Però compte: en una xarxa P2P, la clau ha d'estar a tots dos extrems. Si volem canviar-la, cal reprogramar tots dos.

31.11 Què fer si volem molts nodes

P2P no escala. Si volem més de 2-3 nodes, les opcions són:

1. **P2P amb adreçament manual:** cada node té un ID, el receptor filtra per ID. Funciona per a uns pocs nodes.
2. **LoRaWAN amb ChirpStack autoallotjat:** la millor opció per a més de 2-3 nodes.
3. **Mesh amb Meshtastic:** una altra opció interessant, basada en LoRa, que permet mesh routing.

Per a Hort Osona, si volem créixer, **LoRaWAN amb TTN o ChirpStack** és la millor opció. P2P és per a situacions específiques.

31.12 Quan P2P és la millor opció

Algunes situacions on P2P és millor que LoRaWAN:

- **Hackatges ràpids.** Tens un node, vols transmetre, no tens temps de configurar TTN.
- **Educació.** Per aprendre LoRa a fons, res millor que un P2P.
- **Privacitat extrema.** Si no vols que cap dada passi per tercers.
- **Xarxes molt petites.** 1-2 nodes, no escalable.
- **Proves de camp.** Per validar la cobertura abans de muntar la xarxa completa.

31.13 Migració de P2P a LoRaWAN

Si comences amb P2P i vols migrar a LoRaWAN, la bona notícia és que **el hardware és el mateix**. Només cal canviar el software.

Concretament:

- El node P2P passa a executar LMIC o RadioLib en mode LoRaWAN.
- El receptor SX1262 a la Raspberry ja no cal; el gateway SX1302 s'hi connecta.
- Afegim un gateway LoRaWAN (Waveshare SX1302 HAT) a la Raspberry.
- Configurem TTN o ChirpStack.
- Adapta el payload a CayenneLPP (o JSON).

El temps de migració és d'unes hores, no pas setmanes.

31.14 Resum

En aquest capítol hem après què és LoRa P2P, quan té sentit, i com implementar-lo. Hem vist el codi del node (ESP32 + SX1262 + RadioLib) i del receptor (Python + SX1262 a la Raspberry). Hem après les limitacions d'aquesta arquitectura i quan és millor migrar a LoRaWAN. En el proper capítol veurem les proves de camp: com validar la cobertura, com calibrar el sistema, i com resoldre els problemes habituals.

31.15 Exercicis pràctics

1. Escribe el codi del node P2P amb RadioLib.
2. Escribe el codi del receptor en Python per a la Raspberry.
3. Prova la comunicació a 1 metre. Hauries de rebre el 100% dels missatges.
4. Prova a 50 metres en exterior.
5. Prova a 100 metres amb un obstacle al mig.
6. Mesura el RSSI a diferents distàncies.
7. Compara el rendiment de SF7 i SF12 a 100 metres.
8. Documenta al README els resultats de les proves.

Comandes útils:

```
# Receptor a la Raspberry
python3 ~/homelab/scripts/lora_p2p/receptor_lora_p2p.py
```

```
# Subscriure's a MQTT per veure les dades
mosquitto_sub -h 100.115.134.76 -t "lora/hort/#" -v -u bernat -P CONTRASENYA
```

Paraules clau: **LoRa P2P**, **point-to-point**, **RadioLib**, **SX1262**, **Python**, **receptor**, **ESP32**, **node**, **CayenneLPP**, **payload**, **xifrat**, **AES**, **ACK**, **retransmissió**, **adreçament**, **mesh**, **Meshtastic**, **RSSI**, **SNR**, **packet loss**, **time on air**, **EU868**, **SF7-SF12**, **BW125**, **Antena**, **ad-hoc**, **privat**, **xarxa privada**, **TTN alternatiu**, **ChirpStack**.

Capítol 32 — Proves de camp, cobertura i resolució de problemes

"El dia que posis el node al camp, vindrà una de dues coses: funcionarà o no funcionarà. Aquest capítol és per si no funciona."

32.1 Per què les proves de camp són essencials

En telecomunicacions, hi ha una regla no escrita: **el laboratori menteix, el camp diu la veritat**. Pots tenir el sistema perfectament configurat al taller, amb RSSI de -50 dBm i SNR de 12 dB, i quan el posis al camp amb un arbre al mig, pluja, i un angle d'antena lleugerament diferent, tot canvia.

Per això, les proves de camp són fonamentals. Hem de validar:

1. **Cobertura**: el node arriba al gateway?
2. **Qualitat del senyal**: amb quin RSSI / SNR?
3. **Fiabilitat**: arriba sempre, o hi ha pèrdues?
4. **Banda i data rate**: SF7 funciona, o cal SF9?
5. **Durada de la bateria**: aguanta 1 mes, 3 mesos, 1 any?

32.2 Material per a les proves

Per fer proves de camp, necessitem:

- Un node LoRa (Heltec LoRa 32 V3, per exemple).
- Un gateway funcionant (la Raspberry amb Concentrator).
- Un portàtil amb el monitor sèrie del node.
- Accés a la consola de TTN.
- Una brúixola (per orientar l'antena).
- Un mapa o paper mil·limetrat (per dibuixar la cobertura).
- Un multímetre (per mesurar el voltatge de la bateria).
- Un cable SMA-SMA curt (per connectar l'antena temporalment).
- Una persona al camp (tu, amb el node).
- Una persona a la Raspberry (tu, mirant la consola).
- O tots dos alhora, amb un telèfon per comunicar-se.

32.3 Procediment general de proves

Un procediment sistemàtic per validar la cobertura:

1. **Configura el node** amb un codi que transmet cada 10 segons. Això ens permetrà veure ràpidament si funciona.
2. **Col·loca el node a la posició final** (o el més a prop possible).
3. **Mira la consola de TTN**. Hauries de veure els uplink.
4. **Anota RSSI i SNR** de cada transmissió.
5. **Espera 5-10 minuts** i compta quantes transmissions has rebut.
6. **Calcula el percentatge de pèrdua** = (transmissions enviades - transmissions rebudes) / transmissions enviades.
7. **Mou el node** a una altra posició i repeteix.

A la pràctica, fes un mapa de la zona amb les lectures:

```

      Nord
      ↑
[Node]   Posició 1: RSSI -65, SNR 9.5, pèrdua 0%
         Posició 2: RSSI -85, SNR 6.0, pèrdua 5%
[Gateway] Posició 3: RSSI -55, SNR 11.0, pèrdua 0%
```

32.4 Mètriques clau

A la consola de TTN, podem veure per a cada uplink:

- **RSSI**: potència del senyal rebut. Valors típics:
 - **> -80 dBm**: excel·lent.
 - **-80 a -100 dBm**: bo.
 - **-100 a -110 dBm**: acceptable.
 - **< -110 dBm**: pobre, probablement problemes.
- **SNR**: relació senyal-soroll. Valors típics:
 - **> 10 dB**: excel·lent.
 - **5-10 dB**: bo.
 - **0-5 dB**: acceptable.
 - **< 0 dB**: al límit, problemes.
- **Spread Factor**: el data rate usat.
- **Bandwidth**: amplada de banda del canal.
- **Frequency**: canal usat.

A més, podem veure la **distribució** al llarg del temps: si RSSI baixa gradualment, pot ser un problema (bateria, antena, obstacle). Si baixa sobtadament, pot ser un canvi a l'entorn (un arbre ha crescut, una persona passa per allà).

32.5 Paràmetres a ajustar

Si els resultats no són bons, podem ajustar:

Spreading Factor (SF)

Si volem més abast, pujem SF. Com més alt, més lent però més robust:

- **SF7**: ràpid, curt abast. Per a distàncies curtes amb bona visió.
- **SF9**: bon equilibri. Recomanat per defecte.
- **SF11-SF12**: per a casos extrems. Triga molt i té límit de duty cycle.

A la pràctica, podem començar amb SF7, i si no arriba, pujar a SF9, SF10, etc.

Potència de transmissió

A la consola de TTN, podem veure quina potència està fent servir el node. Podem forçar-la:

- A la consola del node, secció "Network Layer → ADR", podem desactivar ADR.
- A la secció "Network Layer → Transmit Power", podem posar un valor fix (per exemple, 14 dBm).

Si el node té bona visió i està a prop, podem baixar a 8 dBm per estalviar bateria. Si tenim obstacles, pujar a 14 dBm.

Posició de l'antena

L'antena ha d'estar:

- **Vertical** (per a polarització vertical, l'estàndard a EU868).
- **Amb visió directa** al gateway (sense arbres, edificis, muntanyes al mig).
- **Com més amunt millor** (teulada, torre, pal).
- **A 1/4 d'ona del terra** (8.6 cm a 868 MHz) o múltiples d'aquesta distància.

A la pràctica, una antena a 3 metres d'alçada amb visió directa pot cobrir 5-10 km. Una antena a 1 metre d'alçada al costat d'un edifici, 100 metres.

32.6 Optimització de l'antena

A vegades, una bona antena marca la diferència. Algunes millores:

- **Antena més gran** (1/2 ona en lloc de 1/4 ona): +3 dB de guany.
- **Antena de fibra de vidre** amb guany de 5-6 dBi en lloc de 2 dBi: +3 dB.
- **Antena Yagi direccional**: 8-10 dBi, però perd l'omnidireccionalitat.
- **Posició allunyada de metalls**: les estructures metàl·liques absorbeixen el senyal.
- **Cable coaxial de qualitat**: LMR-200 o LMR-400, no RG-58.

A 868 MHz, cada 6 dB de millora equivalen a doblar la distància. Així que +6 dB d'antena = 2x distància.

32.7 Resolució de problemes habituals

El node no es connecta al gateway (no apareix a TTN)

Causes possibles:

1. **Antena no connectada:** revisa-la.
2. **Identificadors incorrectes** (DevEUI, AppEUI, AppKey): comprova'ls.
3. **Freqüència incorrecta:** el node ha d'estar a EU868.
4. **Gateway apagat o no connectat:** revisa la consola de TTN.
5. **Distància massa gran:** apropa el node temporalment.
6. **Problema d'alimentació:** el node es reinicia constantment.

El node es connecta però perd molts missatges

Causes possibles:

1. **RSSI baix:** problema de cobertura. Mou el node o el gateway.
2. **SNR baix:** interferències. Canvia de canal o SF.
3. **Bateria baixa:** el node no pot transmetre amb prou potència.
4. **Antena mal orientada:** prova altres posicions.

El node consumeix massa bateria

Causes possibles:

1. **Deep sleep no funciona:** el node està sempre actiu.
2. **Transmissions massa sovint:** cada 5 min és poc, cada 10 segons és massa.
3. **Wi-Fi o Bluetooth activats:** consumeixen molta energia.
4. **Bateria defectuosa:** canviar-la.
5. **Temperatura extrema:** la bateria dura menys a temperatures baixes.

El gateway no rep transmissions

Causes possibles:

1. **Concentrador no connectat:** revisa els logs.
2. **SPI mal configurat:** revisa els pins.
3. **Freqüència del gateway diferent del node:** han de coincidir.
4. **Antena del gateway mal connectada:** revisa-la.
5. **Problema d'Internet:** revisa que la Raspberry tingui connectivitat.

32.8 Tests automatitzats

Per validar el sistema de forma regular, podem fer tests automatitzats:

```
# Test 1: el gateway està connectat?
curl -s http://100.115.134.76:3001/ | grep -q "concentrator" && echo "Gateway OK" || echo "Gateway FALL"

# Test 2: les últimes dades del node
influx query '
  from(bucket: "hort-osona")
  |> range(start: -1h)
  |> filter(fn: (r) => r._measurement == "sensor_lora")
  |> count()
' --org bernatlab --token TOKEN
```

A Uptime Kuma, afegim un test que comprova que rebem dades cada 15 minuts. Si no, alerta.

32.9 Documents i bitàcoles

Cada vegada que anem al camp, val la pena portar una **bitàcola**:

- Data i hora.
- Posició del node (coordenades GPS si és possible).
- RSSI / SNR.
- Pèrdues de paquets.
- Voltatge de la bateria.
- Condicions meteorològiques (pluja, vent, etc.).
- Notes (obstacles nous, arbres que han crescut, etc.).

Amb el temps, aquesta bitàcola ens permetrà veure patrons i prendre decisions informades (on posar el gateway, quina antena fer servir, etc.).

32.10 Quan el problema és l'entorn

A vegades, la cobertura no és bona perquè l'entorn és difícil:

- **Bosc dens:** els arbres absorbeixen el senyal, sobretot amb fulla (a l'estiu, més pèrdua que a l'hivern).
- **Edificis:** formigó, metall, vidre amb revestiment metàl·lic són hostils.
- **Muntanyes:** difícil salvar-les sense repetidors.
- **Interferències:** altres sistemes a 868 MHz poden molestar.

Solucions:

- **Moure el gateway** a una posició millor (teulada, torre, etc.).
- **Afegir un segon gateway** en una posició intermèdia.
- **Augmentar l'alçada de l'antena.**
- **Canviar a una antena més direccional.**

- **Usar SF més alt** per a més robustesa.

32.11 Quan afegir un segon gateway

Si la zona és gran o hi ha obstacles, un sol gateway pot no ser suficient. Afegir-ne un segon:

- **Rep la mateixa transmissió** (LoRaWAN és broadcast, tots els gateways reben).
- **Millora la cobertura** (redundància).
- **Permet triangular** (a partir del RSSI i temps d'arribada, podem localitzar el node).

A la pràctica, un segon gateway és útil quan:

- La zona té més de 5 km².
- Hi ha obstacles grans (muntanyes, edificis alts).
- Volem molta fiabilitat (sempre volem cobertura).

Al BernatLab, podem afegir un segon gateway a la Raspberry del camp, o a casa d'un veí, o a qualsevol punt amb electricitat i Internet.

32.12 Pla de proves per a Hort Osona

Per validar el sistema a Hort Osona, podem fer aquestes proves en ordre:

Dia 1: proves al taller

1. Connectar el node amb el sensor.
2. Verificar que el node arrenca correctament.
3. Verificar que es connecta a TTN.
4. Mirar els primers uplinks.
5. Mesurar el consum en deep sleep.

Dia 2: proves a curta distància

1. Posar el node a 10 metres del gateway.
2. Verificar que arriba amb bon RSSI.
3. Comptar transmissions durant 10 minuts.
4. Moure el node a 50 metres, 100 metres, etc.

Dia 3: proves al camp

1. Portar el node a l'hort (245 metres de casa).
2. Posar-lo a la posició final.
3. Verificar que arriba al gateway de casa.
4. Mesurar RSSI / SNR.

5. Si cal, moure'l fins trobar una bona posició.
6. Deixar-lo 24 hores i veure com es comporta.

Dia 4: proves de bateria

1. Carregar la bateria al màxim.
2. Posar el node a la posició final amb el deep sleep activat.
3. Cada dia, mirar el voltatge de la bateria.
4. Estimar quants dies dura.

Dia 5: proves de cobertura

1. Portar el node a diferents punts de l'hort i del voltant.
2. Fer un mapa de cobertura.
3. Identificar zones mortes.
4. Decidir si cal un segon gateway o una antena millor.

Dia 6: proves de llarg termini

1. Deixar el node en producció 1 setmana.
2. Mirar les gràfiques d'estabilitat (RSSI, SNR al llarg del temps).
3. Documentar el comportament.
4. Ajustar el que calgui.

32.13 Manteniment continu

Un cop el sistema estigui en producció, cal mantenir-lo:

- **Cada setmana:** mirar les gràfiques d'estabilitat (RSSI, SNR, bateria).
- **Cada mes:** netejar l'antena del node (pols, brutícia).
- **Cada trimestre:** revisar la bateria, substituir-la si cal.
- **Cada any:** revisar l'estat de la caixa estanca, substituir segells si cal.

32.14 Quan re-emplaçar

De vegades, malgrat tots els esforços, una posició no funciona. Si:

- El node perd > 50% de transmissions malgrat SF alt i antena bona.
- El node consumeix la bateria en menys d'1 setmana.
- El node es desconnecta del gateway aleatòriament.

Cal:

- Considerar una **altra posició** (a 50 metres hi ha vegades 30 dB de diferència).
- Considerar un **segon gateway** més a prop.
- Considerar **canviar la tecnologia** (per exemple, NB-IoT si tenim cobertura 4G).

32.15 Resum

En aquest capítol hem après a fer proves de camp: com validar la cobertura, com mesurar la qualitat del senyal, com resoldre problemes, quan afegir un segon gateway, i quin pla de proves seguir. Hem après que el laboratori menteix i el camp diu la veritat, i que cal validar tot amb dades reals.

Aquest és l'últim capítol del Mòdul 3. Si tens el LoRa a les mans, comença pel pla del dia 1 (taller), segueix amb el dia 2-3 (curta distància i camp), i no passis al dia 4-5 (bateria i cobertura) fins que tot funcioni bé. Quan tinguis el node en producció 24/7, podem començar el **Mòdul 4 (IA local amb Ollama)** o afegir un segon node al camp.

32.16 Exercicis pràctics

1. Fes un mapa de cobertura a 50, 100, 200, 500 metres del gateway.
2. Mesura el consum de corrent del node en deep sleep.
3. Comprova el voltatge de la bateria cada dia durant una setmana.
4. Documenta els RSSI / SNR a cada posició.
5. Fes proves amb SF7, SF9, SF11. Compara la cobertura.
6. Si tens un segon gateway disponible, configura'l i veu com millora la cobertura.
7. Escribeu un runbook de resolució de problemes basat en les teves experiències.
8. Comparteix les conclusions al README del projecte.

Paraules clau: **proves de camp, cobertura, RSSI, SNR, packet loss, FSPL, LOS, NLOS, visió directa, antena, polarització, guany, dBi, VSWR, gateway, Concentrator, SX1302, node, deep sleep, bateria, SF, BW, CR, TX power, ADR, propagació, atenuació, interferències, pluja, arbres, edificis, muntanyes, mapa, bitàcola, runbook.**